

LA CONJECTURE DE ROSETTA

Modernisation agentique dirigée par le domaine



La conjecture de Rosetta

Modernisation agentique dirigée par le domaine

Pierre Milet Llobet

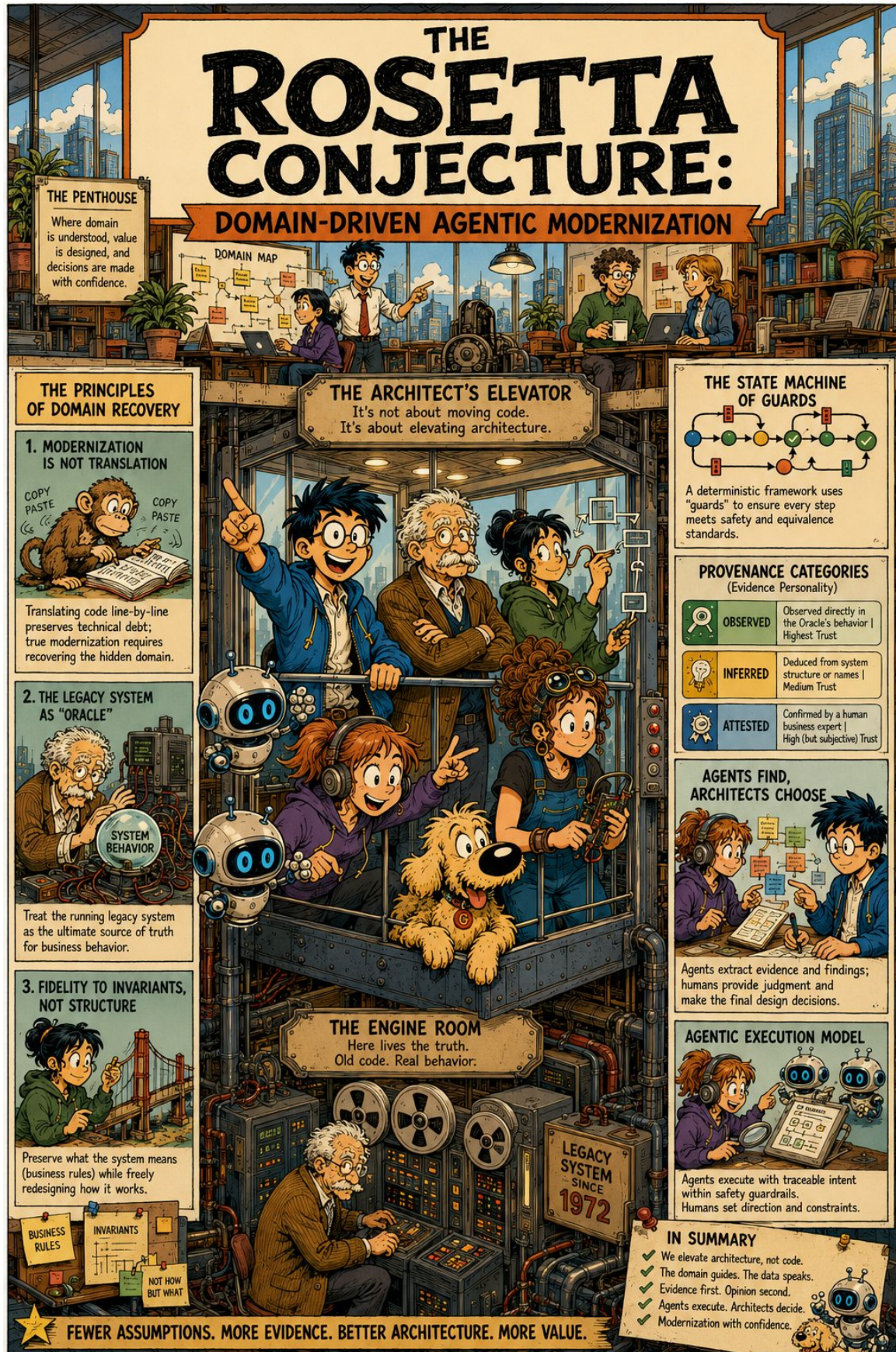
2026

Table des matières

À propos de ce livre	3
Pourquoi Rosetta	3
Prologue. La modernisation qui ne modernise pas	8
I Pourquoi le domaine et non le code	10
1 Le cimetière a des noms	11
2 Le piège de la relocalisation	13
3 Le legacy comme oracle	15
4 Fidélité à l'invariant, non à la structure	17
II L'agent à tous les étages	19
5 L'ascenseur de l'architecte, désormais avec des agents	20
6 Où l'agent finit et où l'architecte commence	22
7 La discipline de provenance	24
8 Le harnais	29
Le substrat déterministe	29
La machine à états	30
Les gardes sont les gates du livre	30
Signaux et observabilité	30
Qui gouverne la machine	31
III Explorer avant de productiser	32
9 Le 3X appliqué à la modernisation	34
10 Récupérer le domaine	36
11 Récupérer, puis trier	39
12 La carte des contextes cible	41
13 La conception tactique du cœur	43

IV Prouver que vous ne l'avez pas cassé	46
14 La chaîne transitive	47
15 Le comportement comme spécification exécutable	50
16 Parité boîte noire et la loi d'ordre	52
V Cohabiter avec deux systèmes	54
17 L'étrangleur, honnête	55
18 La bascule sans foi	57
19 Mettre l'oracle à la retraite sans perdre la référence	59
VI D'Explore à Expand	61
20 Ceci est une hypothèse	62
A Antipatrons	65
B Glossaire	67

Moderniser, ce n'est pas traduire du code. C'est récupérer le domaine.



À propos de ce livre

Pourquoi Rosetta

Pendant plus de quatorze siècles, personne n'a su lire les hiéroglyphes. Les scribes qui les gravaient étaient morts, et avec eux s'était perdu le secret de leur sonorité et de leur sens. L'écriture était toujours là, intacte et muette. Tout était sous les yeux, et l'on ne comprenait rien.

La pierre de Rosette portait le même décret en trois écritures, et l'une d'elles, le grec, se lisait encore. Cela disait ce qui était écrit. Mais savoir ce qui était écrit n'était pas savoir lire l'écriture morte.

La clé fut une autre langue, vivante. L'égyptien n'avait pas tout à fait disparu : il avait continué d'évoluer jusqu'au copte, encore employé dans la liturgie de l'Église copte, écrit avec des lettres grecques et donc lisible. Champollion n'avait aucun scribe pour l'instruire. Il avait la forme vivante et phonétique de la même langue, et c'est par elle qu'il a récupéré le son et le sens de l'écriture morte. La langue restée vivante a livré le secret de celle qui ne l'était plus.

C'est ce que fait cette méthode. Le système hérité est l'écriture cryptique, et ceux qui l'ont écrite sont partis : personne n'a documenté l'intention, les auteurs sont partis à la retraite, le secret s'en est allé avec eux. Mais le système n'est pas tout à fait mort. Il tourne encore, se comporte encore, parle encore chaque jour en production. Ce comportement vivant est votre copte : la forme lisible qui survit à ses auteurs et qui vous laisse récupérer le sens qu'ils ont emporté. Vous n'interrogez pas les scribes. Ils sont partis. Vous interrogez la langue vivante. C'est ce que j'appelle l'oracle.

Vous ne traduisez pas le code dans une autre langue. Vous déchiffrez le domaine que le code a cessé de savoir expliquer.

Et conjecture, parce que c'est ce que j'offre. Non pas une méthode éprouvée contre des centaines de migrations, mais une hypothèse raisonnée sur la manière dont il faudrait procéder quand le domaine est au centre. Une conjecture qui demande à être parcourue, non à être crue.

Ce n'est pas un livre sur les outils. Les outils qu'il mentionne, un compilateur qui réécrit un vieux langage dans un nouveau, un framework qui organise le code en tranches verticales, un graphe qui projette un système sur une structure navigable, un agent qui lit, raisonne et écrit, vieilliront tous. Certains seront obsolètes avant que vous ayez fini de lire. C'est pourquoi ils apparaissent toujours comme des exemples d'une idée, jamais comme l'idée elle-même. Ce que le livre défend, c'est une méthode, et une méthode se mesure aux principes qui la soutiennent, non aux pièces qui l'incarnent aujourd'hui.

Le livre a un centre et deux axes. Le centre, c'est le domaine. Les deux axes sont empruntés. Le premier est l'ascenseur de l'architecte de Gregor Hohpe, qui va du penthouse, où se décide la stratégie, à la salle des machines, où le code s'exécute. Le second est le modèle 3X de Kent Beck, Explore, Expand, Extract, qui décrit comment une idée mûrit, depuis l'intuition jusqu'au standard. Ce livre vit, de façon assumée, dans la phase Explore. Il ne vous vend pas une méthode éprouvée contre des centaines de migrations de production. Il vous offre un argument, cohérent et raisonné, expliquant pourquoi une méthode devrait fonctionner, et il vous invite à la parcourir par cycles rapides d'apprentissage pour découvrir si elle mérite de passer à Expand.

Cette honnêteté est délibérée. Un livre qui en promet trop est démenti par un seul mandat raté. Un livre qui argumente à partir de principes résiste, parce que les principes ne se réfutent pas par des anecdotes. Si dans cinq ans les outils d'aujourd'hui sont de l'archéologie mais que l'argument tient encore debout, le livre aura fait son travail.

Une dernière note sur les exemples. Il y a un système où revenir chapitre après chapitre, et il s'appelle CardDemo. C'est le système d'exemple de modernisation de mainframe qu'AWS publie en libre accès : trente et un programmes COBOL,

avec leurs copybooks, leur batch et leur CICS. Ce n'est pas un jouet inventé pour l'occasion, c'est du vrai COBOL, avec un vrai couplage et au moins une vraie bizarrerie métier. Et c'est un exemple, non le système de production d'une banque, ce qui impose une limite qu'il vaut mieux nommer dès maintenant : ce qui marche ici suggère, sans le prouver, que cela marchera sur un corpus plus vaste et plus sale.

CardDemo gère les comptes de carte de crédit d'un émetteur. Autour du client tournent trois objets : le compte, qui est la ligne de crédit avec sa limite et son solde ; la carte, liée au compte ; et une référence croisée qui réconcilie les trois. Chaque jour, il traite des mouvements contre deux règles qu'il ne laisse jamais se croiser : aucun ne dépasse le crédit disponible du cycle, aucun ne tombe sur un compte expiré. À la clôture du cycle, il calcule l'intérêt en cherchant le taux dans une table de tarification indexée par le groupe du compte. La Figure 1 montre sa carte des contextes telle que la récupération la restitue : un cœur de service des comptes et d'imputation des transactions, un point chaud où le fichier des comptes est réécrit par quatre contextes à la fois, et des coutures propres par où commencer, comme la sécurité, isolée du crédit. Gardez une bizarrerie en tête, car elle reviendra : le contrôle de crédit regarde l'exposition du cycle en cours, non le solde que le client traîne des cycles antérieurs. Cela peut être intentionnel ou un défaut. C'est le métier qui en décide, non le code, et c'est l'exemple parfait de quelque chose que la méthode récupère fidèlement mais qu'elle n'ose pas corriger d'elle-même.

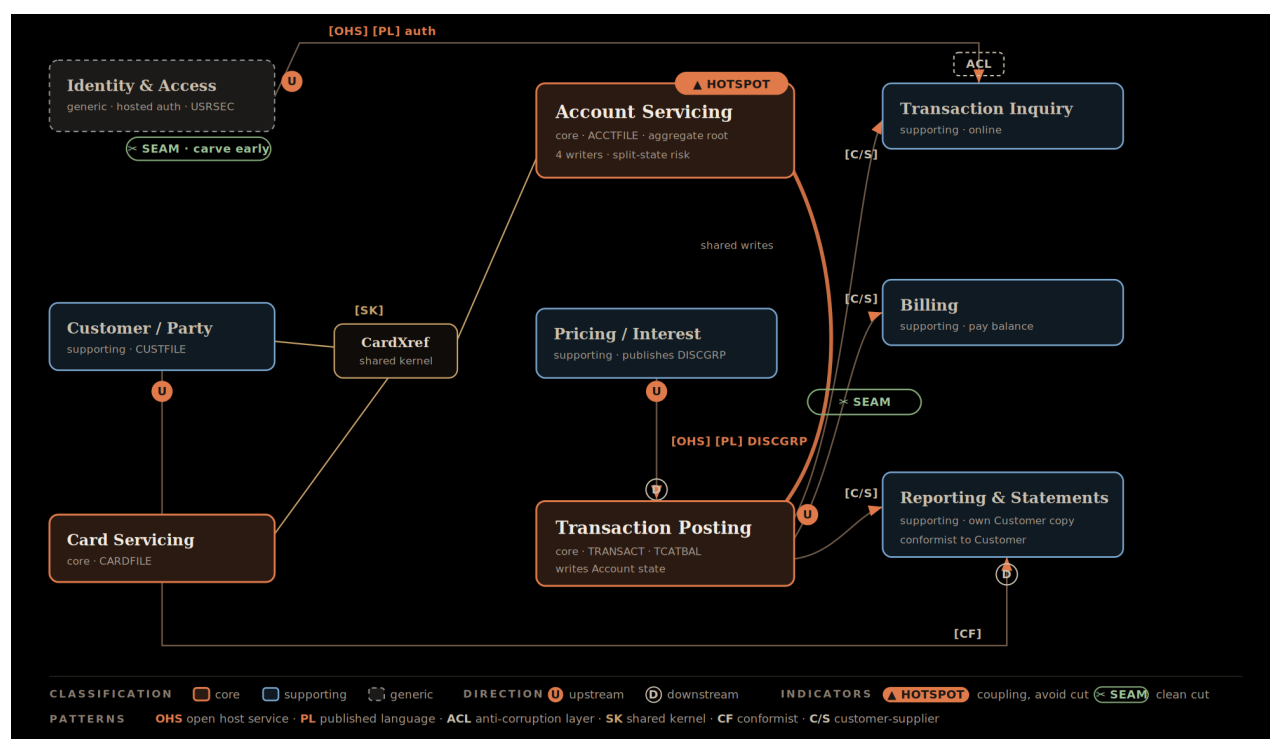


Figure 1: Carte des contextes de CardDemo. Cœur en orange, soutien en bleu, générique en gris ; le point chaud d'écriture partagée sur le fichier des comptes et les coutures de coupe propre.

CardDemo • étape 1/11

CardDemo est l'exemple de modernisation de mainframe que publie AWS en accès libre : trente et un programmes COBOL, avec leurs copybooks, leur batch et leur CICS. Ce n'est pas un jouet inventé pour l'occasion, c'est du vrai COBOL, avec de vrais couplages et au moins une vraie bizarrerie métier.

Il gère les comptes de carte de crédit d'un émetteur. Autour du client tournent trois objets : le compte, qui est la ligne de crédit avec son plafond et son solde ; la carte, liée au compte ; et une référence croisée qui réconcilie les trois. Chaque jour il traite des mouvements contre deux règles qu'il ne laisse jamais se croiser : aucun ne dépasse le crédit disponible du cycle, aucun ne tombe sur un compte expiré. À la clôture du cycle il calcule l'intérêt en cherchant le taux dans une table de tarification indexée par le groupe du compte.

L'enregistrement de compte est déclaré par le copybook CVACT01Y. Ces six champs suffisent à suivre le fil :

01 ACCOUNT-RECORD.

05	ACCT-ID	PIC 9(11).
05	ACCT-CURR-BAL	PIC S9(10)V99.
05	ACCT-CREDIT-LIMIT	PIC S9(10)V99.
05	ACCT-CURR-CYC-CREDIT	PIC S9(10)V99.
05	ACCT-CURR-CYC-DEBIT	PIC S9(10)V99.
05	ACCT-GROUP-ID	PIC X(10).

Remarquez que deux choses coexistent : le solde reporté, ACCT-CURR-BAL, et l'exposition du cycle en cours, ACCT-CURR-CYC-CREDIT moins ACCT-CURR-CYC-DEBIT. Cette coexistence cache la bizarrerie qui reviendra : le contrôle de crédit regarde le cycle, non le solde reporté. Ce peut être intentionnel ou un défaut. C'est le métier qui le décide, pas le code.

Et il convient de le rappeler : c'est un exemple, non le système de production d'une banque. Ce qui marche ici suggère, sans le prouver, que cela marchera sur un corpus plus grand et plus sale.

Entités de CardDemo autour du client

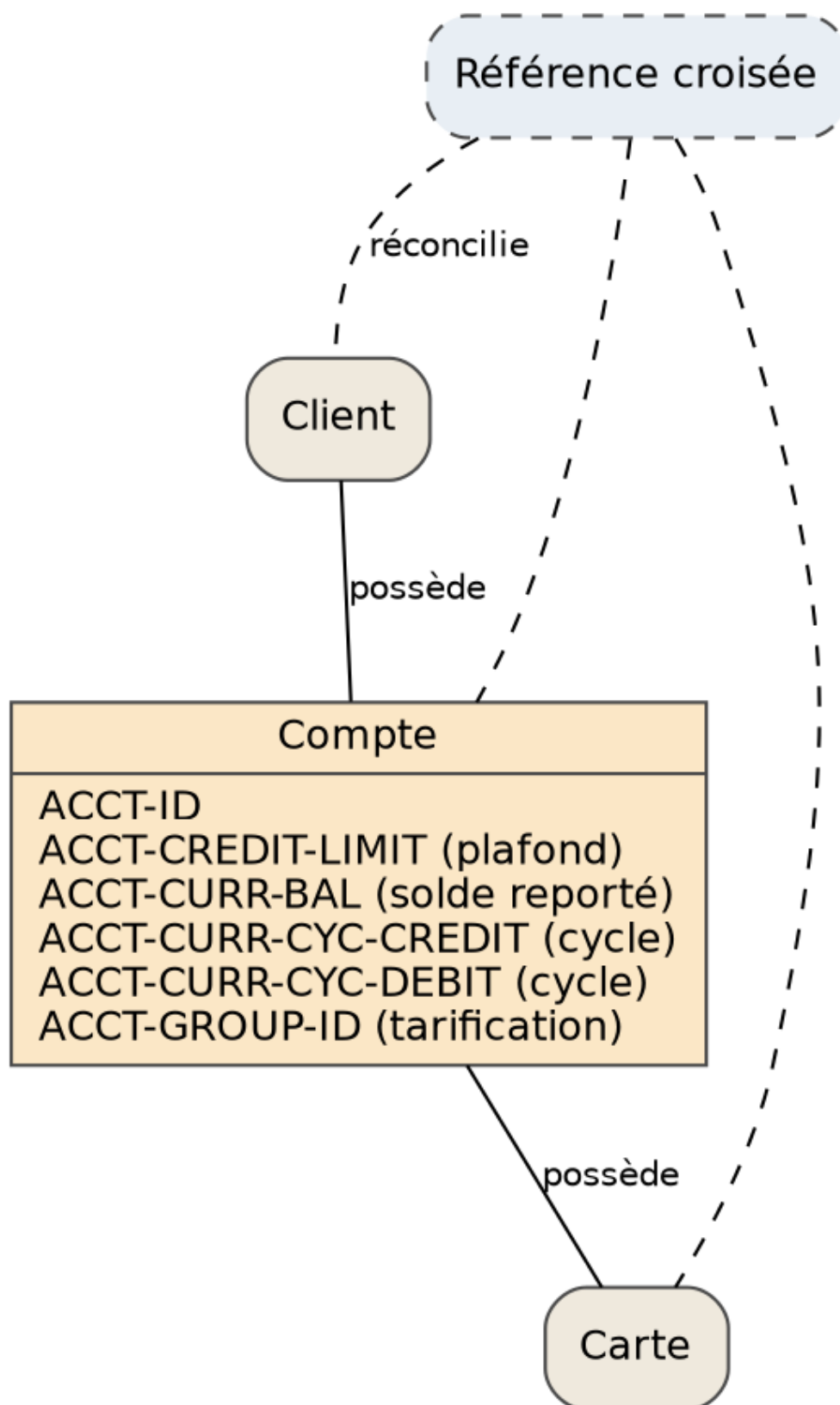


Figure. Les trois entités de CardDemo autour du client : le compte avec son plafond, son solde et son cycle ; la carte liée au compte ; et la référence croisée qui réconcilie les trois.

Prologue. La modernisation qui ne modernise pas

Commencez par ce que ce livre ne dit pas.

Il ne dit pas « utilisez l'IA pour réécrire votre COBOL ». Il soutient presque le contraire. L'argument le plus bruyant de l'industrie en ce moment est que les agents peuvent traduire des millions de lignes de code hérité à une vitesse sans précédent. Cet argument devrait vous inquiéter, car c'est le même pipeline non certifié qui échoue depuis des décennies, seulement plus vite. La vitesse appliquée à une méthode qui ne fonctionne pas ne produit pas de modernisation. Elle produit le prochain post-mortem, plus tôt.

La première contribution de l'IA à la modernisation du mainframe n'est pas de générer du code. C'est de rendre la compréhension abordable, dans des systèmes où elle était devenue impossible. Le but de la méthode de ce livre n'est pas de réécrire. C'est de récupérer le domaine : reconstruire une compréhension véritable de ce que font ces systèmes, quelles capacités et quelles règles métier ils encodent, où se concentre le risque, où vit l'exposition de sécurité et où est l'opportunité. Les décisions stratégiques sur des systèmes qui font circuler ce soir des paiements, des polices et des pensions méritent d'être prises avec compréhension, non avec des conjectures déguisées en feuille de route. Et de presque tout ce que vous récupérez, vous déciderez rationnellement de ne rien réécrire.

Il vaut la peine de situer pourquoi cela importe maintenant. Des décennies de rehosting, de recompilation et de transpilation ont déjà migré le migrable : les systèmes périphériques, les gérables, ceux dont les auteurs sont encore sur la liste de paie. Ce qui reste immergé est tout ce que ces méthodes n'ont pas pu toucher : des monolithes transactionnels avec des décennies de correctifs par-dessus, des auteurs partis à la retraite depuis longtemps, une documentation qui ment. La moitié de l'information critique des grandes organisations vit encore là, et la pression ne cesse de croître : fin de support, coûts de licence qui montent sans retour, régulation qui exige des preuves sur des boîtes noires, experts qui s'en vont sans relâche. L'immersion était, jusqu'à récemment, l'impossible. Ce qui a changé, c'est qu'il entre dans le territoire du possible. Non du facile. Du possible.

Imaginez que vous héritez d'un système. Il est en production depuis des décennies. Il fait circuler de l'argent, ou des vies, ou des cargaisons, quelque chose qui compte. Personne de ceux qui l'ont écrit n'est encore dans l'entreprise. La documentation, si elle existe, ment. Et pourtant le système fonctionne, jour après jour, avec une fiabilité qu'aucun projet récent n'a égalée. Votre mission est de le moderniser.

La première tentation est la plus raisonnable et la plus fautive. Elle consiste à traiter le problème comme une traduction. Vous avez du code dans un vieux langage, vous voulez du code dans un nouveau, alors vous traduisez. Aujourd'hui, des outils le font presque seuls. Un transpileur prend le langage source et émet le langage cible, ligne par ligne, structure par structure. Le résultat compile. Il passe les tests, s'il y en a. Et vous avez échoué, même s'il vous faudra des mois pour vous en apercevoir.

Vous avez échoué parce que vous avez confondu le symptôme avec la maladie. Le problème d'un système hérité n'est pas qu'il soit écrit dans un vieux langage. Le problème est que sa conception, les décisions sur quel concept vit où, quelle règle protège quelle donnée, quelle partie sait quoi d'une autre, a été prise il y a des décennies, s'est érodée à chaque correctif, et n'a jamais été reconsidérée comme un tout. Cette conception, c'est la dette. Traduire le langage conserve la dette intacte. Vous vous retrouvez avec le même enchevêtrement, désormais en syntaxe moderne, et vous avez dépensé un budget à le déplacer. La syntaxe est neuve. La conception est la même qu'avant. Ce n'est pas de la modernisation. C'est de la relocalisation, proprement calligraphiée.

Il y a une variante plus sophistiquée qui tombe dans le même piège. Au lieu de traduire, vous recompilez. Vous prenez le binaire conceptuel du vieux système et vous le faites tourner sur une plateforme neuve, sans toucher au code source. C'est moins cher, plus rapide, moins risqué à court terme. Et ce n'est toujours pas de la modernisation, parce que la conception n'a pas changé. Vous avez changé le sol sous les pieds du système, non le système. C'est une tactique de relocalisation légitime, parfois la bonne, mais l'appeler modernisation, c'est mentir sur ce que vous avez acheté.

Il existe encore une troisième variante, la plus sophistiquée et la plus facile à confondre avec ce que défend ce livre. Au lieu de traduire ou de compiler, elle décompose. Il y a des outils assistés par l'IA qui découpent le monolithe en domaines métier, regroupant programmes et fichiers par leurs dépendances, et refactorisent chaque groupe dans le nouveau langage. Ils parlent de domaines ; certains nomment le DDD.¹ Et pourtant ils restent en deçà, parce que regrouper le code existant en domaines n'est pas la même chose que récupérer le modèle de domaine que le code cache. Un regroupement vous dit quels programmes se touchent entre eux. Il ne vous dit pas quel invariant protège le compte, ni laquelle de ses règles le métier a demandée et laquelle est un accident d'il y a trente ans. Et refactoriser chaque groupe conserve sa conception, exactement comme la traduction, désormais avec des frontières plus nettes autour de la même dette.

Ce livre part d'une affirmation inconfortable. **La modernisation est un problème de domaine, non un problème de code.** Le code est la manière dont le domaine s'est exprimé une fois, sous des contraintes qui n'existent plus. Moderniser pour de vrai, c'est récupérer le domaine que le code cache, le comprendre, et l'exprimer de nouveau dans une architecture qui serve les contraintes d'aujourd'hui. Le langage cible est la partie facile et la moins importante. Le travail est dans le domaine.

Si vous acceptez cela, le reste du livre s'ensuit. Si la modernisation est un problème de domaine, vous avez besoin des outils conceptuels du domaine, et le plus mûr que nous ayons est la conception dirigée par le domaine (DDD), la discipline qui modélise le logiciel à partir du domaine métier et de son langage. Vous devez récupérer le modèle de domaine caché dans un système que personne ne comprend tout à fait, ce qui exige d'interroger le système lui-même au lieu de vous fier à son code ou à sa documentation. Vous devez reconcevoir sans casser, ce qui exige une discipline de vérification qui démontre l'équivalence au lieu de la supposer. Et vous devez cohabiter avec le vieux système pendant que le nouveau naît, parce qu'aucun système qui compte ne s'éteint d'un coup.

Il y a un second fil dans ce livre, et il entre en scène maintenant. Pendant que j'écris ceci, une nouvelle classe d'outil a commencé à changer l'économie de tout ce qui précède. Les agents d'IA, des programmes qui lisent, raisonnent sur ce qu'ils lisent, et agissent, peuvent parcourir un système hérité à une vitesse et une échelle qui étaient auparavant impossibles. Mais les agents ne changent pas la méthode. Ils changent ce que la méthode peut se permettre. Une récupération de domaine qui coûtait jadis des mois d'un expert rare peut désormais coûter des jours, ce qui signifie que vous pouvez vous permettre d'explorer, de vous tromper et de refaire. Des tâches qui appartenaient à la phase Extract, mécaniques et chères, redeviennent Explore, bon marché et réversibles. Voilà la vraie nouvelle. Non que les agents écrivent du code, mais qu'ils rendent l'apprentissage si bon marché qu'ils changent quelles phases du travail vous pouvez vous permettre de parcourir.

Voilà pourquoi ceci est un livre sur le DDD agentique appliqué à la modernisation du legacy. Le domaine au centre. Les agents à tous les étages de l'ascenseur, du penthouse à la salle des machines, augmentant l'architecte à chaque niveau. Et une question qui traverse tout le texte et lui donne son sous-titre. Où l'agent finit et où l'architecte commence. Cette frontière, vous le verrez, n'est pas une affaire de confort ni de confiance. C'est une décision d'architecture, peut-être la plus importante que vous prendrez.

Avant de théoriser pourquoi la traduction échoue, il vaut la peine de regarder ceux qui ont déjà essayé. Le cimetière a des noms.

¹ L'exemple le plus visible au moment où j'écris ces lignes est AWS Transform for Mainframe, présenté comme une capacité d'Amazon Q Developer à re:Invent 2024 et disponible de façon générale depuis mai 2025. Il décompose le monolithe en domaines métier à partir d'un graphe de dépendances et refactorise le COBOL en Java, et il présente la conception dirigée par le domaine comme l'une de plusieurs stratégies de décomposition. L'industrie évolue vite, il convient donc de revérifier l'état de tout outil concret avant de se fier à cette description.

partie I

Pourquoi le domaine et non le code

Chapitre 1

Le cimetière a des noms

Le prologue a soutenu pourquoi traduire échoue. Ce chapitre y met des noms, car le cimetière est réel, il est récent, et ses pierres tombales se lisent.

En avril 2018, TSB a migré plus de cinq millions de clients depuis les systèmes que Lloyds lui hébergeait vers une plateforme neuve, en un seul week-end, déclarée prête sans la preuve pour l'établir. La migration des données a réussi. Le système, non. Près de 1,9 de ses 5,2 millions de clients se sont retrouvés bloqués hors de leurs comptes, certains ont vu les comptes d'autres, et le régulateur britannique a fini par imposer une amende de 48,65 millions de livres pour des défaillances de gouvernance et de gestion du risque, non de matériel. La leçon tient en une phrase. « **La parité des données n'est pas la parité de comportement.** » Déplacer les données intactes ne garantit rien sur ce que le système en fait.

En 2010, le système de paie de Queensland Health est entré en production avec des défauts connus et sans retour en arrière. Quelque 78 000 employés ont été sous-payés, surpayés ou pas payés du tout, pendant des mois. Le coût de la remédiation à la main a été estimé à environ 1,2 milliard de dollars australiens sur des années, et une commission d'enquête l'a classé parmi les pires échecs d'administration publique du pays. Ce ne fut pas une erreur de calcul isolée. Ce fut basculer sans avoir prouvé que le nouveau système se comportait comme il le devait, et sans issue une fois constaté qu'il ne le faisait pas.

En 2023, une grande banque européenne a achevé la migration de douze millions de clients de sa filiale vers sa propre plateforme. Migration « achevée ». Vinrent ensuite des mois de comptes bloqués, de prélèvements rejetés et de clients sans accès à leurs fonds, jusqu'à ce que le superviseur qualifie les perturbations d'inacceptables et installe un contrôleur spécial à l'intérieur de la banque pour forcer la remédiation. « Achevé » n'est pas la même chose que « fonctionne ». De nouveau, la parité des données n'était pas la parité de comportement.

Et puis il y a le verdict. En 2023, l'organe indépendant d'audit informatique de l'État néerlandais a examiné une conversion industrielle de COBOL vers Java à l'organisme qui verse les prestations de chômage et d'invalidité du pays, bâtie sur de la transpilation commerciale de pointe.¹ Le programme l'avait arrêtée, et les auditeurs ont conclu que l'arrêter était la bonne décision, parce que l'approche ne produisait pas de logiciel maintenable. Prenez le verdict au sérieux. Transpiler du COBOL en un Java en forme de COBOL n'apporte presque rien : vous payez une migration et recevez le même système, plus difficile à maintenir, dans un langage qui fait bon effet sur la diapositive mais que ni le spécialiste COBOL ni le spécialiste Java ne peuvent tenir. Si la compréhension manque, aucun changement de syntaxe ne la fournit.

Ces quatre cas ne partagent ni un fournisseur ni une technologie. Ils partagent une racine, et elle n'est pas technique. Elle est de gouvernance. Quelqu'un a affirmé que le système était prêt, ou qu'il était équivalent, au lieu de le prouver, et a basculé sur cette affirmation. La réalité a encaissé la différence. Voilà pourquoi le cimetière importe pour ce livre :

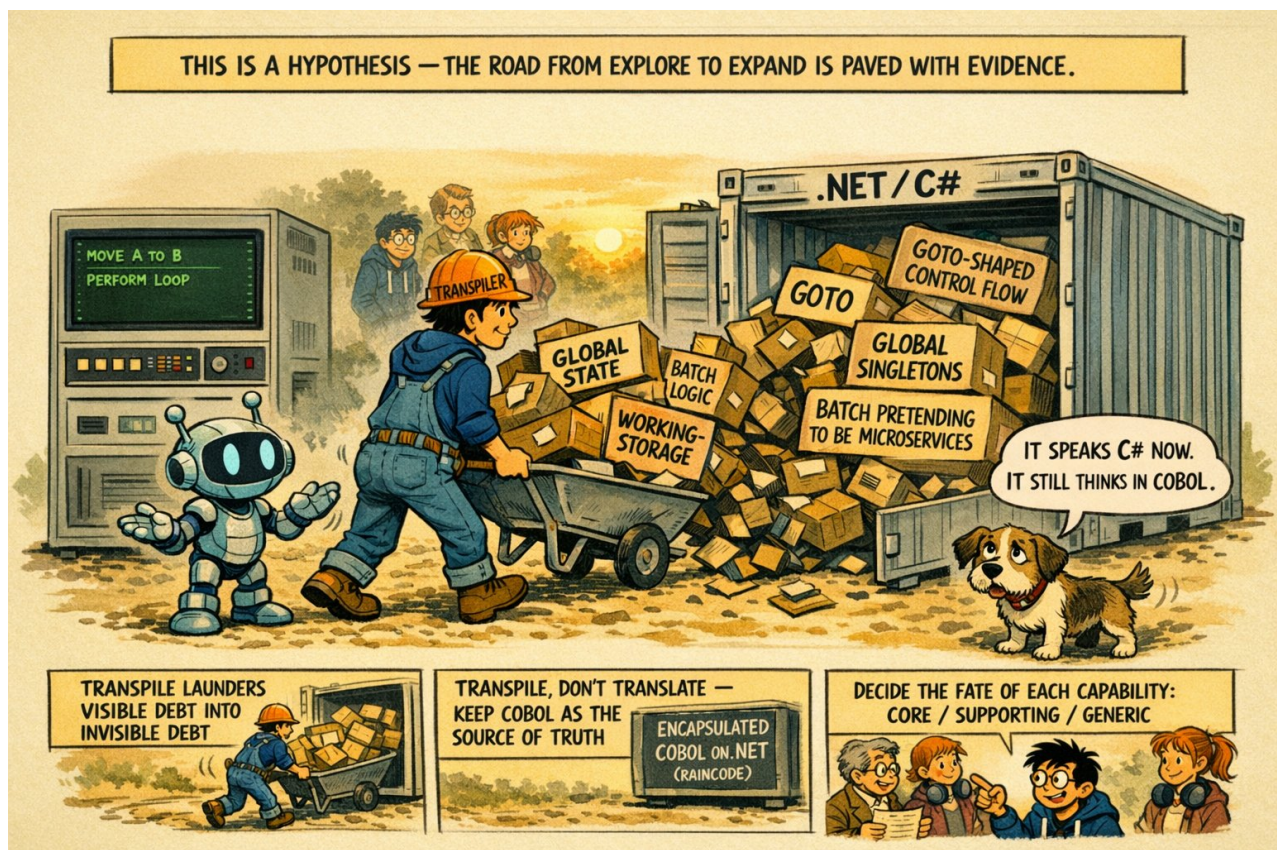
¹Le programme OpenVMS de l'UWV néerlandais (2020-2023) convertissait en Java le code COBOL de deux systèmes de prestations au moyen d'un outil commercial de conversion automatique, avec Capgemini et le sous-traitant Blu Age. L'Adviescollege ICT-toetsing (AcICT), l'organe indépendant d'audit informatique de l'État, a conclu dans son avis du 21 décembre 2023 que l'UWV avait eu raison d'arrêter la conversion industrielle, parce qu'elle ne menait pas à un logiciel maintenable. Sources publiques des quatre cas : les décisions de la FCA et de la PRA (TSB, 2022) ; le rapport de la commission d'enquête Chesterman (Queensland Health, 2013) ; les communiqués de la BaFin (Postbank/Deutsche Bank, 2023) ; et l'avis cité de l'AcICT (UWV, 2023).

le pipeline non certifié que l'industrie veut maintenant accélérer avec des agents est le même pipeline qui a rempli ces tombes. Et la vitesse ne sauve pas une méthode qui ne fonctionne pas ; elle enterre seulement plus vite.

La sortie du cimetière n'est pas un transpileur plus rapide. C'est cesser de se fier aux affirmations, à la documentation et à la mémoire, et commencer à interroger le seul témoin qui ne ment pas, le système vivant. Mais avant cela, il vaut la peine de comprendre précisément pourquoi la traduction échoue, car l'échec est subtil et séduisant. C'est le chapitre suivant.

Chapitre 2

Le piège de la relocalisation



Le code n'est pas la spécification. C'est la preuve.

Il vaut la peine de regarder de près pourquoi traduire échoue, car l'échec est subtil et séduisant. Un transpileur fait exactement ce qu'il promet. Il prend une structure du langage source et produit son équivalent dans le langage cible. Une boucle devient une boucle. Une condition devient une condition. Une routine devient une méthode. La correspondance est fidèle, et c'est là le problème. La fidélité structurelle est précisément ce que vous ne voulez pas conserver.

Pensez à ce qu'il y a dans un programme hérité typique. Il y a de la logique métier, les règles qui définissent ce que le système signifie, pourquoi un montant est rejeté, quand une opération est valide. Et il y a du mécanisme, la plomberie qui fait fonctionner cette logique sur la plateforme de son époque, comment on ouvre un fichier, comment on passe un bloc de mémoire d'un programme à un autre, comment on gère une erreur d'entrée-sortie. Dans le vieux système, logique et mécanisme sont entrelacés, parce que la plateforme de l'époque obligeait à les entrelacer. Il n'y avait pas d'autre moyen.

Quand vous traduisez ligne par ligne, vous traduisez les deux ensemble. Le vieux mécanisme, qui n'existait que pour satisfaire une plateforme que vous abandonnez, voyage vers le nouveau système déguisé en logique. Vous vous retrouvez avec du code moderne qui ouvre des fichiers comme s'il était encore sur la vieille plateforme, qui passe des blocs de mémoire entre modules comme si la mémoire était rare, qui gère des erreurs avec des codes numériques qui ne signifient plus rien. Vous avez importé la plomberie d'une maison que vous avez démolie. Et comme c'est désormais en syntaxe moderne, c'est plus difficile à repérer, parce que cela ressemble à du nouveau code légitime.

La conception subit le même sort. Si le vieux système entassait trois responsabilités distinctes dans un même module parce que les créer coûtait cher, la traduction produit un nouveau module avec les trois responsabilités. Si deux concepts qui auraient dû être séparés partageaient une structure de données par économie, ils continuent de la partager. Les mauvaises frontières sont préservées avec une fidélité parfaite. Le couplage est conservé. La cohésion absente reste absente. La seule chose qui a changé, c'est la couleur de la syntaxe.

C'est ici que beaucoup de gens bien intentionnés se défendent. Ils disent, d'accord, traduisons d'abord et refactorisons ensuite. D'abord un équivalent fidèle, puis on nettoie. Cela paraît prudent. C'est un piège plus profond, pour deux raisons.

La première est économique et psychologique. Une fois que vous avez un nouveau système qui compile et fonctionne, le budget du nettoyage s'évapore. Le projet est déclaré terminé. L'enchevêtrement traduit reste, désormais béni en tant que base de code moderne, et les dix années de maintenance suivantes le traiteront comme sacré. La refactorisation qui devait venir ensuite ne vient jamais. Ce qui devait être une étape intermédiaire devient la destination.

La seconde raison est plus technique et plus intéressante. Refactoriser exige de savoir quel comportement vous devez préserver pendant que vous changez la structure. Si vous avez traduit sans comprendre le domaine, vous n'avez pas ce savoir. Vous ne savez pas laquelle de ces trois responsabilités du module est de la logique métier qui doit survivre et laquelle est du mécanisme qui doit mourir. Vous n'avez pas de filet de sécurité qui vous dise si votre nettoyage a cassé quelque chose. Refactoriser à l'aveugle sur du code que vous ne comprenez pas, ce n'est pas de la modernisation, c'est de la roulette. Nous y reviendrons avec soin, car le filet de sécurité, la preuve que vous n'avez rien cassé, est l'un des piliers de la méthode.

La recompilation, l'autre voie, est honnête à sa manière et donc moins dangereuse, pourvu que vous ne mentiez pas à son sujet. Faire tourner le vieux système sur une plateforme neuve sans toucher au code est une relocalisation. Elle a des usages légitimes. Elle vous sort d'un matériel mourant, vous retire une dépendance à un fournisseur, vous achète du temps. Ce qu'elle ne fait pas, c'est moderniser, parce que le domaine reste exprimé exactement de la même façon, avec les mêmes mauvaises frontières et le même mécanisme entrelacé. L'erreur n'est pas de recompiler. L'erreur est de croire que vous avez fini. Et il vaut la peine d'anticiper quelque chose que le livre défendra plus loin : la recompilation a deux usages pleinement légitimes, comme instrument, l'oracle du chapitre trois, et comme destination délibérée pour ce que vous décidez de ne pas reconcevoir, le tri du chapitre onze. L'illégitime n'est pas de recompiler. C'est de le vendre comme modernisation du cœur.

Il y a un principe qu'il vaut mieux fixer dès maintenant, car il reviendra. **Changer la plateforme et changer la conception sont deux sauts distincts, et ils ne doivent jamais se faire en même temps.** Si vous recompilez pour changer de plateforme, faites-le comme un mouvement propre et isolé, sans toucher à la conception. Si vous reconcevez, faites-le sur une plateforme stable, sans la déplacer sous vos pieds. Qui tente les deux sauts dans le même mouvement se retrouve sans point d'appui pour diagnostiquer ce qui a échoué quand quelque chose échouera, et quelque chose échouera. Ce double saut interdit est l'une des erreurs les plus chères et les plus fréquentes, et il a son propre chapitre plus loin.

Alors, si ce n'est ni traduire ni recompiler, qu'est-ce que moderniser. La réponse courte est récupérer le domaine et l'exprimer de nouveau. La réponse longue occupe le reste du livre. Mais il y a un virage préalable, presque un changement d'attitude, sans lequel rien d'autre ne fonctionne. Vous devez cesser de voir le vieux système comme un boulet dont s'échapper et commencer à le voir comme la source de vérité la plus fiable dont vous disposez. C'est le chapitre suivant.

Chapitre 3

Le legacy comme oracle

Le système hérité a mauvaise réputation, et en partie il l’a méritée. Il est rigide, opaque, cher à maintenir, impossible de recruter des gens qui le connaissent. Mais il y a une chose que ce système fait mieux que n’importe quel document, n’importe quel expert et n’importe quelle spécification que vous pourriez écrire à son sujet. Il sait exactement ce qu’il fait. Il le prouve chaque jour, en production, sans se tromper sur les cas rares, ceux dont vous ne soupçonnez même pas l’existence.

C’est l’idée qui change tout. **Le vieux système n’est pas seulement un boulet. C’est un oracle.** Un oracle, au sens que nous lui donnons ici, est une source qui répond avec la vérité à une question sur le comportement. Vous lui donnez une entrée, il vous donne la sortie correcte, correcte par définition, parce que c’est ce que le système fait vraiment et ce que le métier accepte comme correct depuis des décennies. Vous n’avez à croire personne sur la manière dont une majoration se calcule dans le cas limite, ni sur ce qui se passe quand deux opérations arrivent dans l’ordre improbable. Vous le demandez au système et il vous le dit.

Cela réordonne les priorités de la récupération. La tradition dit que pour comprendre un vieux système on lit son code et sa documentation. Le code est la meilleure des deux sources, mais il est traître, parce qu’il mêle logique et mécanisme, parce qu’il est plein de chemins morts qui ne s’exécutent plus jamais, et parce que le lire vous dit ce que le système semble faire, non ce qu’il fait. La documentation est pire, parce qu’elle décrit ce que quelqu’un a voulu un jour que le système fasse, et le système a passé des années à diverger de cette intention sans que personne ne mette à jour le document. Les deux sources vous donnent des hypothèses. L’oracle vous donne des faits.

Le changement pratique est énorme. Au lieu de discuter pendant des semaines de ce que signifie une routine inintelligible, vous l’exécutez avec des entrées choisies et vous observez. Au lieu de vous fier au souvenir qu’un expert a de la règle du cas spécial, vous générez le cas spécial et vous regardez la réponse. Le vieux système devient un laboratoire. Vous lui faites des expériences. Les questions qui produisaient des réunions produisent maintenant des observations.

Pour que l’oracle soit vraiment utile, il lui faut deux choses, et c’est ici qu’entrent les outils, comme exemples d’un concept. La première est qu’il réponde vite. Un oracle qui met une nuit à vous donner une réponse ne sert pas à explorer, parce que l’exploration a besoin de cycles courts, demander, observer, ajuster la question, redemander, des dizaines de fois par jour. C’est pourquoi il est intéressant d’avoir une copie du vieux système qui tourne en local, en millisecondes, non sur le mainframe de production avec sa file de travaux. Une façon d’y parvenir est de recompiler le vieux système vers une plateforme moderne que vous pouvez exécuter sur votre propre machine, dans un conteneur, autant de fois que vous le voulez. Le compilateur qui fait cette recompilation est un exemple d’outil. Le concept durable, c’est l’oracle rapide et local, une réplique fidèle du vieux comportement qui répond dans le temps d’un cycle de pensée.

Remarquez quelque chose d’élégant. La recompilation, qui au chapitre précédent était une fausse modernisation si vous la confondiez avec la destination, est ici exactement le bon outil pour un autre dessein. Vous ne recompilez pas pour livrer un système. Vous recompilez pour vous construire un oracle. La même technique, méprisée comme but, est précieuse comme instrument. La différence est dans l’usage que vous en faites.

La seconde chose dont l’oracle a besoin, c’est la fidélité. Il doit se comporter comme l’original, non lui ressembler. Si votre réplique diverge du système réel dans les cas rares, elle vous mentira précisément là où vous lui ferez le plus confiance.

C'est pourquoi la fidélité de la réplique ne se suppose pas, elle se valide, en la comparant au système réel sur suffisamment de cas. Un oracle non vérifié est une rumeur à bonne réputation.

Il y a une nuance honnête qu'il vaut mieux ne pas cacher. L'oracle ne sait que ce qu'on lui demande. Il connaît le comportement sur les entrées que vous lui donnez, non sur celles auxquelles vous ne pensez pas. Si un chemin du vieux système ne s'exécute jamais dans vos expériences ni dans le trafic réel que vous observez, son comportement reste non capturé, et c'est un comportement qui peut compter le jour où l'on s'y attend le moins. L'oracle n'élimine pas l'incertitude, il la borne et la rend visible. Cela aura des conséquences quand nous parlerons des entrées à choisir pour l'interroger, car bien choisir les questions est la différence entre un oracle qui couvre les frontières où vivent les bogues et un oracle qui ne fait que confirmer ce qui se passe déjà tous les jours.

Pour l'instant, retenez le virage d'attitude. Le système qu'on vous a demandé de remplacer est, tant qu'il vit, votre meilleur professeur. Avant de le mettre à la retraite, vous devez apprendre de lui tout ce dont vous aurez besoin pour toujours, car lorsqu'il s'éteindra, cette source de vérité disparaîtra. Une bonne part de la méthode consiste à presser l'oracle avec discipline pendant qu'il est encore allumé.

Chapitre 4

Fidélité à l'invariant, non à la structure

Si le vieux système est l'oracle et que ce que vous poursuivez est de préserver son comportement, une question piègeuse surgit. Préserver quoi, exactement. Car si la réponse était de tout préserver, aucune modernisation ne serait possible, vous seriez tenu de conserver jusqu'au dernier détail du vieux système, y compris l'enchevêtrement que vous vouliez supprimer. La fidélité mal comprise vous ramène au piège de la traduction.

La bonne réponse exige une distinction qui est le cœur conceptuel de ce chapitre et l'une des clés de toute la méthode. Il y a deux choses très différentes qu'un vieux système contient, et une seule d'entre elles est sacrée.

La première, c'est le quoi. Les règles métier, les invariants, les règles qui doivent toujours tenir, la sémantique de chaque opération. Un montant ne peut jamais dépasser une certaine limite. Une opération sur une entité expirée est rejetée. Un certain calcul arrondit d'une certaine manière et le résultat compte jusqu'à la dernière décimale parce que c'est de l'argent. C'est l'espace du problème. Il existe indépendamment de la manière dont le vieux système a choisi de l'implémenter. Il survivrait même si vous réécriviez le système de zéro dans n'importe quel langage. C'est ce que vous devez préserver avec une fidélité absolue.

La seconde, c'est le comment. La structure concrète que le vieux système a choisie pour satisfaire ces règles. Quel module contenait quelle logique, quelle structure de données partageaient quels concepts, dans quel ordre on faisait les choses, où étaient tracées les frontières. Que le fichier des comptes soit réécrit par quatre programmes distincts, comme dans CardDemo, est du mécanisme de la vieille solution : le métier n'a jamais demandé quatre rédacteurs, il a seulement demandé que le solde tombe juste. C'est de l'espace de la solution, et c'est une vieille solution, prise sous des contraintes qui n'existent plus. Elle n'est pas sacrée. Elle est, de fait, justement ce que vous venez changer.

Voici la phrase qu'il vaut la peine de graver. **La fidélité va aux invariants et à la sémantique, non à la structure.** Vous préservez ce que le système signifie. Vous êtes libre de reconcevoir la manière dont il y parvient. Cette liberté n'est pas une permission mineure, c'est le prix entier de la modernisation. Sans elle, vous seriez condamné à copier la vieille conception. Avec elle, vous pouvez redessiner les frontières, séparer ce qui était mêlé, donner à chaque concept sa place, et garantir malgré tout que le système continue de signifier la même chose.

Une image aide. Aucune carte du monde n'est fidèle à tout à la fois. La projection de Mercator est fidèle aux caps, la ligne de cap constant sort droite, et c'est pourquoi elle sert à naviguer ; en échange, elle déforme l'aire, et le Groenland paraît aussi grand que l'Afrique alors qu'il est quatorze fois plus petit. Une projection équivalente est fidèle à la surface et déforme les formes. Ce n'est pas un défaut des cartographes. C'est impossible : une sphère ne s'aplatit pas sans rompre quelque chose, et Gauss l'a démontré. Toute carte plane choisit à quoi être fidèle et laisse le reste se déformer.

Moderniser, c'est faire ce choix. Vous choisissez d'être fidèle à la sémantique, à ce que le système signifie, et vous laissez la structure changer de forme. Le vieux système était une carte fidèle à une vieille machine, comme Mercator l'est à la navigation. Le nouveau système sera une autre carte du même domaine, fidèle à autre chose. Le monde, le métier, est le même. Ce qui change, c'est ce à quoi vous êtes fidèle en le dessinant.

Cette distinction a une conséquence qui semble surprenante et qui sera plus loin un pilier technique. Si ce que vous préservez est la sémantique et non la structure, alors deux systèmes peuvent être équivalents en comportement et complètement différents à l'intérieur. Le nouveau système peut avoir d'autres frontières, d'autres modules, un autre

ordre, et donner pourtant la même réponse à la même question. Cela signifie que la preuve que vous n'avez rien cassé ne peut pas consister à comparer les entrailles des deux systèmes, parce que par conception elles sont différentes. Elle doit consister à comparer ce qui s'observe de l'extérieur, le comportement, non l'implémentation. C'est la graine de la parité boîte noire, c'est-à-dire vérifier l'équivalence en ne regardant que les entrées et sorties observables, jamais l'intérieur. Nous la développerons en entier dans la Partie IV.

La distinction éclaire aussi où se cachent les erreurs les plus dangereuses d'une modernisation. Il y a des pièces d'un vieux système qui ressemblent à du mécanisme mais qui portent une décision métier à l'intérieur. L'exemple classique est la gestion d'une erreur technique. Quand le vieux système ne trouve pas une donnée et réagit d'une certaine manière, cette réaction est-elle de la pure plomberie, détecter une absence, ou est-ce une règle métier déguisée, décider qu'une absence équivaut à une certaine valeur par défaut avec des conséquences réelles ? Si vous la traitez comme du mécanisme et la jetez, vous avez silencieusement perdu une règle. Si vous la traitez comme une règle et la conservez, vous traînez de la plomberie. Distinguer laquelle est laquelle ne peut se faire par la seule lecture, parce que le code entrelace les deux choses. Cela se fait en demandant à l'oracle quelle conséquence métier a ce comportement. La perte silencieuse de sémantique, conserver la coquille technique et perdre la règle qu'elle cachait, est l'un des antipatrons les plus chers, et il figure dans son annexe.

Il vaut la peine de clore par une image qui résume les trois idées de cette première partie. Pensez au domaine récupéré comme à un modèle que vous pourrez habiller de bien des architectures différentes. Le modèle, les règles et la sémantique, est unique et stable. Les architectures possibles sur ce modèle sont nombreuses. La traduction échoue parce qu'elle copie une vieille architecture sans récupérer le modèle. L'oracle vous permet de récupérer le modèle avec des faits au lieu de rumeurs. Et la fidélité bien comprise vous dit ce qui du modèle est intouchable, la sémantique, et ce qui est libre, la structure. Avec cela en main, la question cesse d'être comment je traduis ce code et devient laquelle des nombreuses architectures possibles sur ce domaine sert le mieux les contraintes d'aujourd'hui. C'est une question d'architecte. Et les architectes, il se trouve, travaillent désormais accompagnés d'agents.

CardDemo • étape 2/11

Dans CardDemo, le fichier des comptes est réécrit par quatre programmes différents, deux en batch et deux en ligne :

```
*   CBTRN02C   comptabilisation des mouvements (batch)
*   CBACT04C   calcul de l'intérêt                (batch)
*   COACTUPC   création et édition de compte    (CICS)
*   COBIL00C   paiement de facture              (CICS)
      REWRITE FD-ACCTFILE-REC FROM ACCOUNT-RECORD
```

Le métier n'a jamais demandé quatre rédacteurs ; il a seulement demandé que le solde tombe juste. Les quatre rédacteurs sont un mécanisme de la vieille solution, non la sémantique du domaine. Cette répartition de l'écriture est le point chaud qui reviendra quand nous récupérerons la carte.

partie II

L'agent à tous les étages

Chapitre 5

L'ascenseur de l'architecte, désormais avec des agents

Gregor Hohpe a proposé une métaphore qui sert à comprendre ce que fait un architecte, et que nous allons ici étirer un peu au-delà de son propos d'origine. Imaginez une organisation comme un immeuble. Au penthouse se décide la stratégie, vers où va le métier, ce qui compte. Dans la salle des machines, bien des étages plus bas, le code s'exécute et les choses se passent pour de vrai. Entre les deux, il y a des étages intermédiaires, l'architecture, les équipes, les processus. La plupart des gens vivent à un seul étage. Ceux du penthouse parlent de vision et ne savent pas ce qu'il y a dans les binaires. Ceux de la salle des machines connaissent chaque bit et ne savent pas pourquoi il compte. La valeur de l'architecte, dit Hohpe, c'est qu'il prend l'ascenseur. Il monte et descend. Il traduit la stratégie en structure et la structure en stratégie. Il connecte des étages qui, sinon, ne se parleraient pas.

Dans une modernisation, l'ascenseur est littéral. Au penthouse, il y a les questions du domaine, quels concepts ce métier possède, quelles règles le gouvernent, où sont les frontières naturelles entre ses parties. Dans la salle des machines, il y a le vieux système, son code, ses structures de données, ses routines entrelacées. Et le travail de modernisation est précisément un voyage d'ascenseur, descendre à la salle des machines récupérer ce que le système fait, remonter au penthouse comprendre ce que cela signifie en termes de domaine, s'arrêter à l'étage intermédiaire décider l'architecture, et redescendre vérifier. Qui ne sait que lire du code ne peut pas moderniser, parce qu'il ne voit pas le domaine. Qui ne sait que le domaine ne le peut pas non plus, parce qu'il ne peut rien récupérer d'un système qu'il ne sait pas lire. La modernisation est un métier d'ascenseur.

Ici entre le second fil du livre, et il vaut mieux le dire avec précision pour qu'il ne vieillisse pas. Une classe d'outil est apparue, les agents, qui peut parcourir cet ascenseur. Un agent, au sens qui importe ici, est un système capable de lire une grande quantité de matière, de raisonner dessus, et de produire des résultats, du code, des modèles, des analyses, des propositions. Ce n'est ni de la magie ni un collègue. C'est un outil doté d'une propriété nouvelle et importante. Il opère à tous les étages. Il peut lire du code dans la salle des machines. Il peut proposer des frontières de domaine au penthouse. Il peut générer l'échafaudage d'une architecture aux étages intermédiaires. Avant, chaque étage exigeait un outil différent et une personne différente. Maintenant il existe une classe d'outil qui les parcourt tous.

C'est ce qui justifie l'adjectif agentique du titre, et il vaut la peine de bien comprendre pourquoi. Ce n'est pas que les agents soient un accélérateur ponctuel d'une tâche concrète, comme un compilateur plus rapide. C'est que les agents sont présents à chaque niveau du travail, augmentant l'architecte à tous les étages de l'ascenseur à la fois. L'architecte qui descendait jadis seul à la salle des machines descend maintenant avec un outil qui lit dix fois plus vite et ne se fatigue pas. Celui qui montait au penthouse proposer une carte de contextes délimités (bounded contexts) monte maintenant avec un outil qui a déjà lu tout le système et apporte des candidats. Le DDD reste au centre, intact, parce que le DDD est la discipline qui met le domaine en premier, et cela, les agents ne l'ont pas inventé et ne le remplaceront pas. Ce qui change, c'est que désormais le domaine se récupère, se modélise et se vérifie avec un compagnon à chaque étage.

Il faut résister à deux erreurs opposées à ce sujet. La première est le dédain, croire que les agents sont de l'autocomplétion avec des prétentions et que la méthode ne change en rien. La seconde est la fascination, croire que les agents font le travail et que l'architecte est superflu. Les deux erreurs échouent au même endroit, elles ne comprennent pas ce qui change

exactement. Ce qui change, ce ne sont pas les principes, c'est l'économie. Et quand l'économie d'une activité change suffisamment, quelles activités vous pouvez vous permettre change, et cela altère bel et bien la méthode, non dans ses principes mais dans sa portée.

L'exemple le plus clair est donné par le modèle 3X de Beck lui-même, que nous verrons en détail plus loin. Beck distingue trois phases dans la vie d'une idée. Explore, quand vous cherchez ce qui marche et que presque tout ce que vous essayez se jette. Expand, quand quelque chose marche et qu'il faut le mettre à l'échelle. Extract, quand c'est déjà stable et que vous en pressez l'efficacité. Dans une modernisation traditionnelle, récupérer le domaine se traitait comme s'il s'agissait d'Extract : un acte final, cher, lent, fait par des experts rares, qu'on faisait une fois et qu'on tenait pour bon même s'il était mauvais, parce que le refaire était impensable. Avec un agent qui récupère en jours ce qui coûtait des mois, cette même tâche baisse de coût jusqu'à devenir Explore. Vous pouvez vous permettre de récupérer le domaine, de le regarder, de ne pas être convaincu, de le jeter et de le récupérer autrement. Ce qui était un engagement irréversible devient une expérience bon marché. Voilà la transformation profonde. Non que l'agent récupère le domaine, mais qu'il le rend si bon marché que le récupérer cesse d'être un acte unique et solennel pour devenir un cycle d'apprentissage rapide.

Ainsi l'agent prend l'ascenseur avec l'architecte, à tous les étages, et rend le voyage si bon marché qu'il change quels voyages vous pouvez vous permettre. Mais il y a un étage où l'agent ne décide pas, il propose seulement. Savoir où est cette frontière, et pourquoi elle doit exister, c'est le chapitre suivant, et c'est la décision la plus importante du livre.

Chapitre 6

Où l'agent finit et où l'architecte commence

Si l'agent parcourt tous les étages, la question naturelle et dangereuse surgit. Alors, que fait l'architecte que l'agent ne fait pas. La réponse paresseuse est ce que l'agent ne sait pas encore faire, et elle est paresseuse parce qu'elle vieillit à chaque nouvelle version de l'agent et parce qu'elle traite la frontière comme une limitation technique temporaire. La bonne réponse est différente et ne vieillit pas. Il y a une classe de décisions que l'architecte doit prendre non parce que l'agent ne le peut pas, mais parce qu'elles ne doivent pas être déléguées, et comprendre pourquoi, c'est comprendre la méthode.

Commençons par la manière dont raisonne un agent, car la clé est là. Un agent ne raisonne pas bien sur du code source brut. Le code est long, répétitif, plein de détail non pertinent, et il entrelace logique et mécanisme. Ce qu'un agent fait bien, c'est raisonner sur des projections du système, des représentations qui distillent une dimension du système et écartent le bruit. Un graphe qui montre quoi appelle quoi. Un modèle qui montre quelle donnée coule vers quelle décision. Une structure qui montre quelles règles protègent quelle entité. Sur ces projections, l'agent raisonne avec puissance, parce que la projection a déjà fait le travail de séparer le signal du bruit. Le code est le territoire, et la projection est la carte. Plus loin, nous prendrons du recul et verrons que le code lui-même est aussi une carte, celle que ses auteurs ont dessinée d'un territoire plus profond, le métier ; pour l'instant, ce niveau suffit. Et les agents, comme les explorateurs, travaillent mieux avec des cartes.

Cela dessine déjà une première frontière. Construire de bonnes projections, décider quelle dimension du système mérite une carte, est un travail d'architecte. L'agent navigue la carte, mais quelle carte dessiner pour quelle question est une décision de conception. Une mauvaise carte mène un agent très puissant à une conclusion fausse avec une totale assurance, ce qui est la pire classe d'erreur.

Mais la frontière de fond est plus profonde et tient à la différence entre trouver et choisir. Quand vous récupérez un domaine d'un vieux système, il y a des choses que vous découvrez et des choses que vous décidez. Vous découvrez qu'une certaine règle existe, parce qu'elle est là, dans le comportement de l'oracle, et que vous pouvez la pointer. C'est une trouvaille. Elle a une source. Un agent peut le faire, parce que cela consiste à observer et à rapporter ce qui s'observe. En revanche, quand vous décidez que deux concepts que le vieux système mêlait doivent aller dans des contextes séparés, vous ne l'avez trouvé nulle part. Le vieux système ne vous l'a pas dit. Vous l'avez choisi, à la lumière de votre jugement sur ce qui sert le mieux le métier d'aujourd'hui. C'est une décision de conception. Elle n'a pas de source dans le vieux système, parce que sa source, c'est vous.

La frontière entre l'agent et l'architecte est la frontière entre trouver et choisir. L'agent trouve. Il rapporte ce que le système fait, avec sa source, parce que trouver, c'est observer. L'architecte choisit. Il décide quelle architecture, quelles frontières, quelles règles consolider ou séparer, parce que choisir est un acte de jugement qui ne se déduit d'aucune observation. Et la raison pour laquelle cette frontière ne doit pas être franchie n'est pas que l'agent soit incapable de proposer une architecture, bien sûr qu'il peut en proposer une. La raison est qu'une proposition sans source, présentée comme si c'était une trouvaille, contamine le modèle avec des décisions déguisées en faits. Et un modèle où vous ne pouvez pas distinguer ce que vous avez récupéré du système de ce que vous avez inventé est un modèle auquel vous ne pouvez vous fier pour rien d'important, parce que vous ne savez pas combien il est vrai à propos du métier et combien il est l'opinion d'un outil.

Il y a un vieux contraste qui donne son nom à cette frontière. Aristote plaçait la connaissance dans l'observable, dans ce qu'on voit le monde faire. Platon la plaçait dans les idées, dans ce que les choses doivent être, ce qui ne se lit nulle part parce que ce n'est pas dehors, cela s'atteint en pensant. L'oracle est aristotélicien : il dit ce que le système fait, et il le dit parce que cela s'observe. L'architecte, quand il choisit, est platonicien : il décide ce que le système doit être, et cela ne se trouve ni dans le code ni dans l'oracle, non parce qu'il n'a pas bien regardé, mais parce que ce n'est pas là. Trouver, c'est observer ce qu'il y a. Choisir, c'est raisonner ce qu'il devrait y avoir. Voilà pourquoi un choix ne peut jamais porter l'étiquette de trouvaille. Il ne vient pas du même endroit.

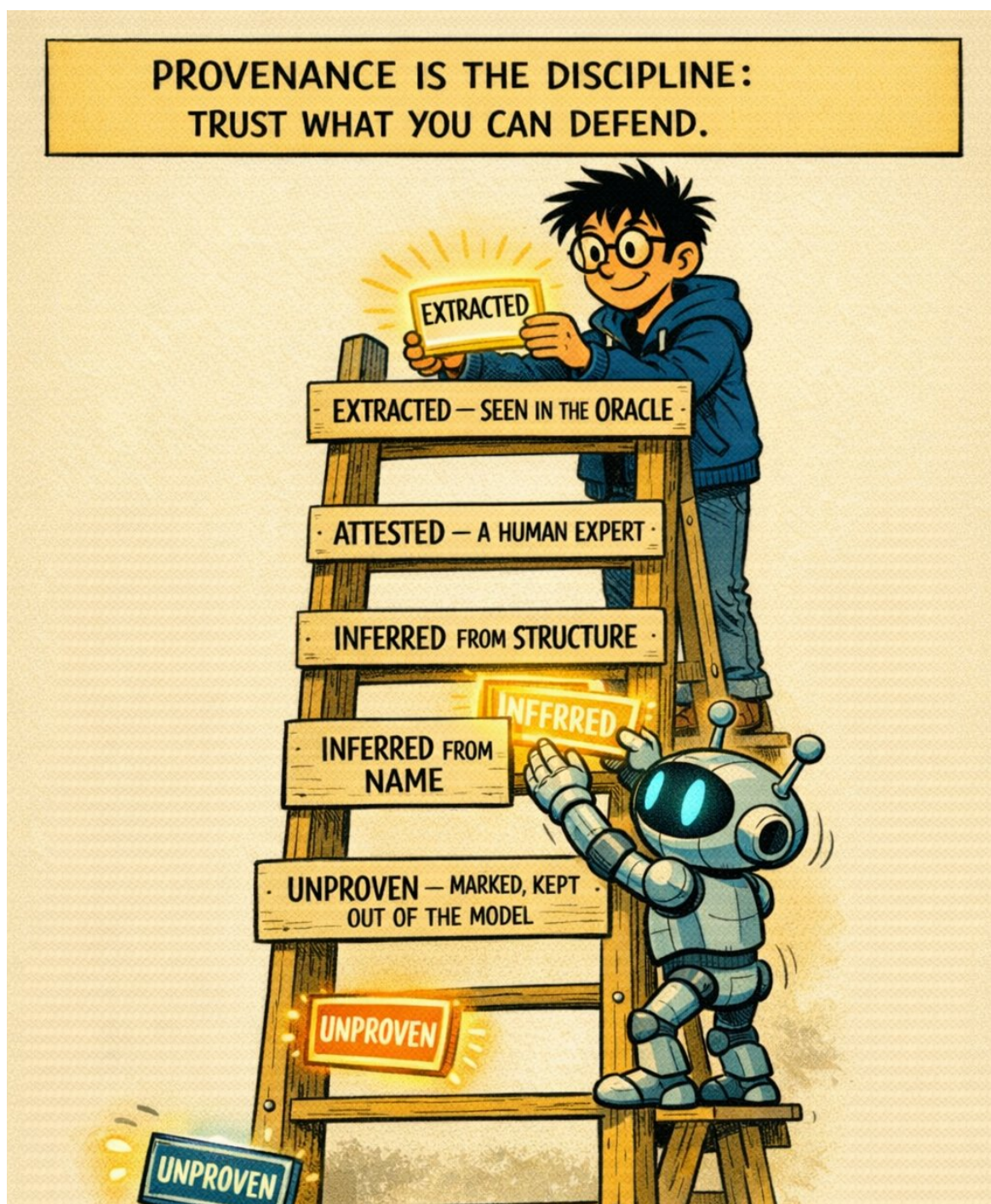
Cela fait de la frontière une décision d'architecture, non de confort. Vous ne dites pas l'agent va jusqu'ici parce qu'au-delà il se trompe. Vous dites l'agent trouve et l'architecte choisit parce que mêler les deux détruit la fiabilité du modèle. C'est une frontière épistémique, sur quelle sorte de connaissance est chaque chose, et il se trouve qu'elle coïncide exactement avec la frontière de qui fait quoi. Le récupéré, qui a une source, l'agent peut l'apporter. Le décidé, qui n'a d'autre source que le jugement, c'est l'architecte qui doit le poser, et cela doit rester marqué comme tel.

Il vaut la peine d'insister sur une asymétrie que les gens sous-estiment. L'agent peut proposer des options de conception, et il le doit, parce que là il est très utile. Le bon procédé est que l'agent, après avoir récupéré le domaine, présente à l'architecte un éventail d'architectures possibles, chacune avec ses preuves et ses conséquences, en particulier ses conséquences sur la vérification ultérieure. Cette option est moins chère à tester mais consolide deux règles en une. Cette autre est plus fidèle mais plus chère à vérifier. L'agent est excellent pour dresser cette table d'options. Ce qu'il ne peut pas faire, c'est s'asseoir à la table et choisir pour vous, parce que choisir, c'est engager le jugement, et le jugement répond au métier, non à un outil. L'agent sert le menu. L'architecte choisit.

Il reste une question pratique. Si le récupéré et le décidé doivent se distinguer, comment se distinguent-ils en pratique, à l'intérieur du modèle, de sorte que quiconque le lit sache ce qu'il regarde. La réponse est une discipline simple et rigoureuse, et c'est le chapitre suivant.

Chapitre 7

La discipline de provenance



Un modèle de domaine récupéré d'un vieux système est un mélange de choses d'origines très différentes. Certaines, vous les avez extraites directement du comportement observé. D'autres, vous les avez inférées de la structure, elles paraissent vraies mais vous ne les avez pas confirmées. D'autres, un expert métier qui est depuis trente ans avec le système vous les a confirmées. D'autres sont des décisions de conception que vous avez prises vous-même. Et d'autres sont simplement des conjectures que vous n'avez pas encore prouvées. Elles cohabitent toutes dans le même modèle, et de l'extérieur elles se ressemblent, des phrases qui décrivent le domaine. Cette apparence d'égalité est un danger, parce qu'elle vous invite à leur faire confiance également, alors qu'elles méritent des confiances très différentes.

La solution est une discipline que je vous demande de prendre au sérieux même si elle sonne bureaucratique, parce que c'est elle qui soutient la fiabilité de toute la méthode. **Aucune affirmation n'entre dans le modèle sans déclarer sa provenance.** Chaque règle, chaque frontière, chaque invariant que vous écrivez porte une marque qui dit d'où il vient et, donc, combien vous pouvez vous y fier.

Les catégories de provenance sont peu nombreuses et claires. Il y a des affirmations extraites, observées directement dans le comportement de l'oracle, les plus fiables, parce que vous les avez vues se produire. Il y a des affirmations inférées de la structure, déduites de la façon dont le système est organisé, plausibles mais non confirmées, parce que la structure peut tromper. Il y a des affirmations inférées du nom, suggérées par la manière dont une chose s'appelle, les plus faibles, parce que les noms dans les vieux systèmes mentent souvent. Il y a des affirmations attestées par un expert, qu'un humain connaissant le métier confirme. Et il y a des affirmations non prouvées, des conjectures marquées honnêtement comme telles pour que personne ne les prenne pour des faits. Les marques concrètes que vous employez importent peu, ce qui importe, ce sont les distinctions qu'elles capturent. CardDemo donne l'exemple parfait. Le contrôle de crédit regarde l'exposition du cycle en cours et non le solde que le client traîne d'avant. Ce comportement est extrait, vous l'avez vu se produire dans l'oracle. Mais s'il s'agit d'une règle délibérée ou d'un défaut vieux de décennies, c'est une affirmation non prouvée, et cette part-là, la méthode ne la décide pas : elle l'emmène au registre de décisions pour que le métier la résolve.

Il y a deux ou trois subtilités qui méritent qu'on s'y arrête, car c'est là que la discipline montre sa valeur. La première. Ce qui est attesté par un expert n'est pas la même chose que ce qui est extrait de l'oracle, même si les deux paraissent fermes. L'expert peut mal se souvenir, peut décrire comment il croit que le système fonctionne au lieu de comment il fonctionne, peut décrire une version d'il y a dix ans. Son témoignage est précieux, mais c'est un témoignage, non une observation. C'est pourquoi il porte sa propre marque, distincte de l'extrait. Quand le témoignage de l'expert et l'observation de l'oracle divergent, la discipline de provenance vous avertit que vous avez un conflit à résoudre, au lieu de vous laisser choisir à l'aveugle. Et presque toujours l'oracle gagne, parce que le système fait ce qu'il fait, quoi que dise le souvenir.

Il vaut la peine, en outre, de distinguer deux experts que le mot cache. L'expert du métier habite l'espace du problème : il sait pourquoi la règle existe et ce qui arriverait si elle changeait. Le vétéran du mainframe, celui qui maintient le système depuis vingt ans, habite l'espace de la solution : il sait quel tour du code est un idiome de la plateforme et lequel cache une décision, où sont enterrés les cas qui mordent, et si la réplique qui fait office d'oracle se comporte comme l'original là où cela compte le plus. Son témoignage est le canal attesté pour désambiguïser le mécanisme de la logique, exactement la frontière où le chapitre quatre a situé les erreurs les plus chères. Les agents ne le déplacent pas : ils l'élèvent. Il cesse de dépenser ses années dans la maintenance triviale et passe à faire ce que lui seul peut faire, interpréter l'oracle.

La seconde subtilité relie ce chapitre au précédent et boucle l'idée de la frontière. Il y a une règle d'or. **Ce qui ne peut citer aucune source n'est pas une trouvaille, c'est un choix, et il doit quitter le modèle du récupéré pour un registre de décisions.** Si vous écrivez une affirmation sur le domaine et qu'en essayant de marquer sa provenance vous découvrez que vous ne pouvez pas pointer d'où elle vient, que vous ne l'avez ni extraite ni inférée ni fait attester par quiconque, alors vous ne décrivez pas le vieux système. Vous décidez quelque chose sur le nouveau. Cette affirmation n'appartient pas au modèle du récupéré. Elle appartient au registre des décisions de conception, qui est une tout autre affaire, l'intrant de la reconception, non une part du portrait du domaine.

Cette règle est exactement la frontière du chapitre précédent, trouver contre choisir, rendue opérationnelle. La provenance est le mécanisme par lequel la frontière entre l'agent et l'architecte cesse d'être une intention et devient une pratique que vous pouvez auditer. Quiconque peut prendre le modèle et demander, de chaque affirmation, d'où elle vient. Si tout le récupéré a une source et que tout ce qui n'a pas de source est dans le registre de décisions, la frontière est propre. Si vous trouvez des affirmations sans source camouflées parmi les récupérées, vous savez que quelqu'un, un humain ou un agent, a glissé un choix déguisé en trouvaille, et vous savez que le modèle est contaminé juste là.

Certains diront que c'est trop de cérémonie pour un travail d'ingénierie. La réponse est que le coût de la cérémonie est

petit et le coût de ne pas l'avoir est énorme et différé. Sans discipline de provenance, six mois plus tard personne ne se souvient si la règle du cas limite vous l'avez observée dans le vieux système ou quelqu'un l'a inventée en réunion, et cette différence est la différence entre un fait du métier et une opinion peut-être fausse. La provenance est bon marché à maintenir si vous la maintenez dès le début, et ruineuse à reconstruire si vous ne l'avez pas fait. C'est, au fond, la même leçon que la vérification que nous verrons plus loin. Mieux vaut démontrer que supposer, et mieux vaut savoir d'où vient chaque chose que se fier à ce que quelqu'un s'en souviene.

Avec la frontière tracée et la provenance en place, nous avons les deux moitiés de la méthode agentique. Mais une frontière qu'il faut se rappeler et une discipline qu'il faut bien vouloir suivre sont fragiles. Ce qui les soutient quand personne ne surveille n'est pas la bonne volonté, c'est une machine. Et de cette machine traite le chapitre qui clôt cette partie.

CardDemo • étape 3/11

Le contrôle de crédit vit dans CBTRN02C, paragraphe 1500-VALIDATE-TRAN. Il regarde l'exposition du cycle, non ACCT-CURR-BAL :

```

COMPUTE WS-TEMP-BAL = ACCT-CURR-CYC-CREDIT
                    - ACCT-CURR-CYC-DEBIT
                    + DALYTRAN-AMT
IF ACCT-CREDIT-LIMIT >= WS-TEMP-BAL
  CONTINUE
ELSE
  MOVE 'OVERLIMIT TRANSACTION'
  TO WS-VALIDATION-FAIL-REASON-DESC
END-IF

```

Ce comportement est extrait : vous l'avez vu se produire dans l'oracle. Savoir si c'est une règle délibérée ou un défaut vieux de décennies est, en revanche, une affirmation non prouvée. La méthode ne la tranche pas : elle la porte au registre de décisions pour que le métier la résolve.

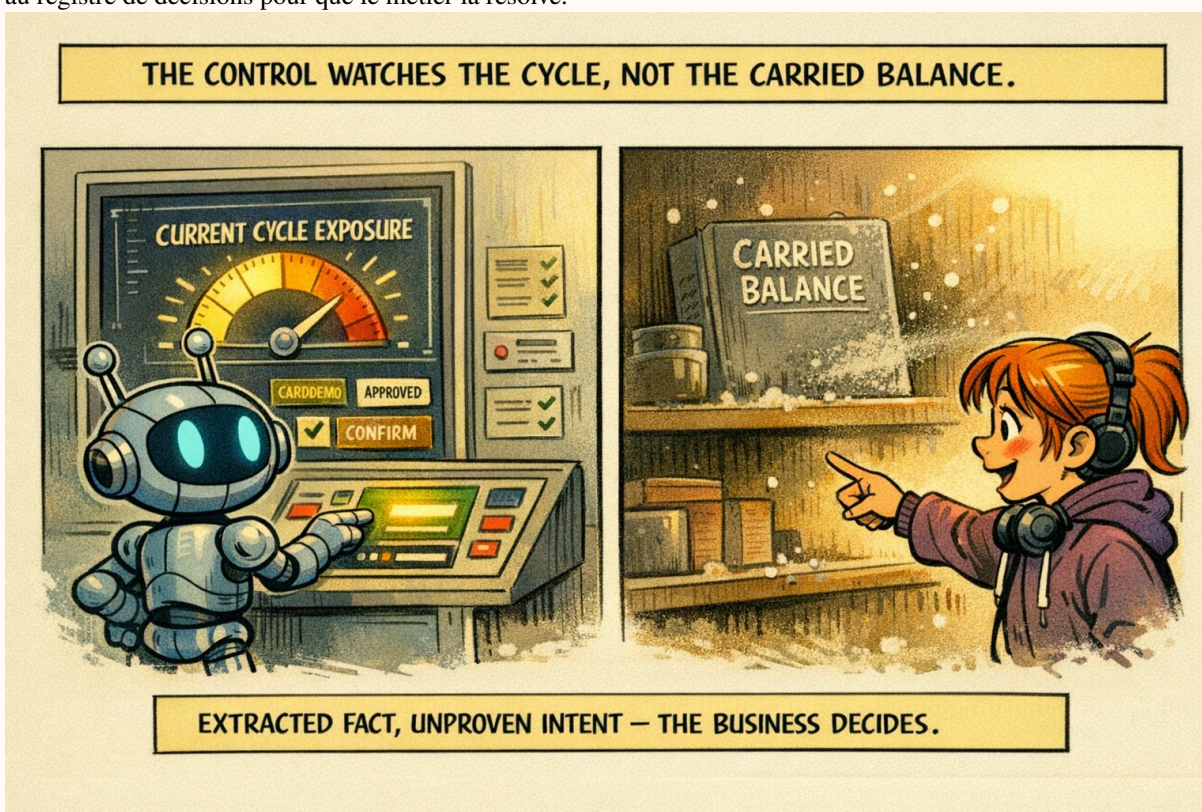


Figure. La bizarrerie du crédit sur une ligne de temps de deux cycles de facturation : le contrôle regarde l'exposition

du cycle en cours et ignore le solde reporté du cycle précédent.

Chapitre 8

Le harnais

Les deux chapitres précédents ont tracé une ligne et demandé une discipline. La ligne sépare ce que l'agent trouve de ce que l'architecte choisit. La discipline exige que chaque affirmation déclare sa provenance. Les deux sont justes, et les deux sont fragiles, parce qu'elles dépendent de quelqu'un qui se souvient de les respecter. Une ligne qui n'existe que dans la tête de celui qui l'a tracée se franchit le premier jour où le calendrier presse. Une discipline qui ne tient que par la bonne volonté s'abandonne à la première urgence.

La question de ce chapitre est ce qui fait que la ligne et la discipline tiennent quand personne ne surveille. **La réponse, c'est que ce n'est pas la volonté qui les soutient, c'est une machine.** J'appelle harnais cette machine. Ce n'est pas un outil concret. C'est une manière d'organiser le travail de l'agent pour que la frontière et la provenance cessent d'être des intentions et deviennent des propriétés du système, vérifiables et difficiles à contourner.

Le substrat déterministe

Au chapitre six, il a été dit que l'agent ne raisonne pas bien sur du code brut, mais sur des projections, des cartes qui distillent une dimension du système. Une question est restée sans réponse, et c'est la plus importante de toutes. Si l'agent raisonne sur la projection, qui construit la projection, et pourquoi s'y fier.

Il y a une réponse tentante et fausse : que ce soit l'agent qui la construise, en lisant le système et en dessinant la carte. Elle est fausse parce qu'alors la projection hériterait de la faillibilité de l'agent. Un agent qui dessine la carte qu'il va ensuite interpréter peut mal tracer une arête, inventer un appel qui n'existe pas, en omettre un qui existe, puis raisonner avec une totale assurance sur une carte qui ment. L'erreur la plus chère n'est pas l'agent qui se trompe en lisant une carte. C'est l'agent qui se trompe en la dessinant et que personne ne le remarque.

C'est pourquoi la projection n'est pas inventée par l'agent. **La projection se dérive mécaniquement du système, par un processus déterministe qui n'a pas d'opinion.** On analyse le code et les données avec des règles fixes, et on verse le résultat dans une structure navigable : quoi appelle quoi, quelle donnée coule vers quelle décision, quelle condition protège quel état. Ce versement est reproductible. Si vous le lancez deux fois sur le même système, vous obtenez la même carte. Et chaque élément de la carte peut pointer de quelle partie du système il est issu, ce qui est la provenance du chapitre précédent appliquée au substrat : chaque arête cite son origine dans le code.

Sur ce substrat déterministe, l'agent fait ce qu'il fait bien. Il cherche, met en relation, propose. Une recherche par le sens trouve des candidats : cette structure ressemble à un agrégat, cette condition ressemble à un invariant. Et le graphe déterministe vérifie si ces candidats tiennent, si l'appel existe vraiment, si la donnée coule comme on le croit. La recherche propose, le graphe dispose. L'agent ne vérifie jamais contre sa propre mémoire, qui invente avec aplomb. Il vérifie contre le substrat, qui n'invente pas.

En pratique. Le substrat se matérialise souvent comme un graphe de propriétés interrogé aux côtés d'un index sémantique. Cet appariement, une recherche sémantique qui propose et un graphe qui vérifie, est ce que l'industrie appelle GraphRAG. La machine à états peut être une bibliothèque légère d'états et de transitions. Les agents de chaque état sont coordonnés par un framework d'orchestration. Les noms concrets changeront.

Ce qui ne change pas, c'est l'idée : substrat déterministe, états à accès borné, gardes qui ne se contournent pas.

La machine à états

Le substrat rend fiable ce que l'agent lit. Il reste à rendre fiable ce que l'agent fait. Récupérer un domaine n'est pas un seul acte, c'est une séquence : lire, proposer, confronter à l'oracle, ratifier, décider l'architecture, vérifier. Si cette séquence est laissée au jugement de l'instant, elle se désordonne. On ratifie avant de confronter. On décide l'architecture sur un modèle à demi récupéré. On vérifie ce qui n'est pas encore décidé.

La solution n'est pas une idée neuve. C'est la plus vieille façon qu'a l'informatique d'être fiable : une machine à états. La machine de Turing, le modèle abstrait par lequel commence la théorie de la computation, est exactement cela, une machine dont le prochain pas est déterminé par son état et son entrée, sans marge pour la surprise. Et c'est, point par point, le contraire d'un agent, qui, devant la même entrée, peut répondre des choses différentes. Le harnais vit de ce contraste : il met l'imprévisible, l'agent, à travailler à l'intérieur du prévisible, la machine.

Le harnais organise la séquence comme cette machine à états. **Chaque état a un seul but, l'agent qui le poursuit, le contexte dont il a besoin et rien de plus, et un ensemble borné d'outils qu'il peut atteindre.** Un état pour récupérer la tactique. Un autre pour confronter les candidats à l'oracle. Un autre pour ratifier. Un autre pour proposer l'architecture. L'agent qui récupère n'atteint pas les outils qui décident, parce que dans cet état il ne décide pas. Celui qui vérifie ne peut pas toucher le modèle, parce que dans cet état il ne modélise pas. Le contexte de chaque état se nettoie à l'entrée, pour que le bruit d'une tâche ne contamine pas la suivante.

Ce n'est pas de la bureaucratie. C'est ce qui empêche les tâches de se mêler. Un agent avec accès à tout et mémoire de tout tend à tout faire à la fois, et tout faire à la fois est exactement ainsi qu'un choix se déguise en trouvaille et qu'une conjecture se glisse comme un fait. La machine à états est la frontière du chapitre six faite topologie : l'agent ne peut pas passer de trouver à choisir parce que ce sont des états différents, à accès différents, et le pas entre eux est gardé.

Les gardes sont les gates du livre

Entre état et état, il y a une garde. Une garde est une condition déterministe qui doit tenir pour que la machine avance. Ce n'est ni un conseil ni une liste que quelqu'un passe en revue. C'est un verrou. Si la condition ne tient pas, la transition n'a pas lieu, et il n'y a aucun moyen de la forcer à la main sans laisser de trace.

Ici le harnais rassemble des pièces qui, dans d'autres chapitres, apparaissent éparses. **Ce que le livre appelle gates sont les gardes de cette machine, non des listes de bonnes intentions à la fin d'une branche.** La ratification est une garde : ne passe au modèle que ce qui peut défendre sa provenance, et le douteux ne transite pas, il reste ou se confronte de nouveau. La loi d'ordre, que nous verrons dans la Partie IV, est une garde : on n'entre pas dans l'état de refactorisation de qualité tant que la garde d'équivalence n'est pas au vert. La bascule, dans la Partie V, est une garde : on ne coupe pas tant que ses conditions ne sont pas réunies, équivalence, couverture, ombre et divergences résolues.

Quatre disciplines qui paraissaient indépendantes se révèlent être la même chose vue en des endroits différents : des conditions de transition d'une machine qui ne laisse pas avancer tant qu'elles ne sont pas réunies. Voilà la différence entre une méthode qui vous demande de faire les choses dans l'ordre et une méthode où le désordre est, tout simplement, impossible.

Signaux et observabilité

Une garde ne vaut que ce que vaut l'information avec laquelle elle décide. Si la garde d'équivalence demande « le vieux et le nouveau coïncident-ils » et que la réponse est un oui ou un non sans nuance, elle ne sert à rien. Elle a besoin de savoir sur quoi ils coïncident, avec quelle couverture, quels cas ont été testés et lesquels non. Cette information, ce sont les signaux : ce que le système émet sur lui-même pendant qu'il travaille.

L'observabilité n'est pas un luxe opérationnel ajouté à la fin. C'est ce qui permet aux gardes de décider sur des bases solides et à la frontière d'être auditable. **Chaque décision de la machine laisse une trace avec sa provenance, de sorte que quiconque peut demander après coup pourquoi on a avancé, et la machine répond.** Pourquoi cette règle a été

ratifiée. Avec quelle preuve cette équivalence a été close. Quelle couverture avait la spécification quand la coupe a été autorisée.

La couverture mérite un mot, car c'est elle qui met un chiffre sur le résidu honnête. Au chapitre de l'oracle, il a été dit que le vieux système ne sait que ce qu'on lui demande, et que ce qui n'est pas exercé reste non capturé. La couverture est la mesure de combien a été exercé. Elle n'élimine pas le résidu, aucun processus ne capture ce qui ne s'est jamais manifesté, mais elle le borne et le rend visible. Une garde qui s'ouvre avec une couverture pauvre ne certifie pas l'équivalence, elle certifie qu'ils ont coïncidé là où vous avez regardé. L'observabilité est ce qui empêche de confondre les deux.

Qui gouverne la machine

Il reste à dire où est l'humain. La tentation, quand la machine fonctionne, est de le mettre dans la boucle, approuvant chaque pas, ou de l'en sortir tout à fait et de la laisser tourner seule. Les deux sont des erreurs.

L'architecte n'est pas dans la boucle, il est sur la boucle. Il n'approuve pas chaque transition, cela, ce sont les gardes déterministes qui le font, elles sont là pour ça et le font mieux qu'une personne fatiguée un vendredi. L'architecte surveille la machine, lit ses traces, et décide aux points où la machine, par conception, ne décide pas : où est la frontière, quelle architecture, quoi faire de ce qui n'a pas de provenance. **L'agent opère à l'intérieur de la machine. L'architecte gouverne la machine. Ce ne sont pas le même métier et ils ne se délèguent pas l'un à l'autre.**

Cela clôt l'argument de la Partie II. L'agent trouve, et la machine garantit que seul ce qui a une source entre comme trouvaille. L'architecte choisit, et la machine garantit que le choix est marqué comme choix et ne se déguise pas en autre chose. La frontière n'est plus une intention qu'il faut se rappeler. C'est la forme de la machine.

Avec la frontière tracée, la provenance en place et le harnais qui les fait respecter, les deux moitiés de la méthode agentique s'emboîtent : l'agent qui trouve et l'architecte qui choisit, séparés par une machine qui ne les laisse pas se confondre. Nous avons le comment. Il manque une question d'attitude, celle qui décide quand il convient d'utiliser cela sérieusement et quand non, et pourquoi une méthode jeune traitée comme mûre échoue. C'est là qu'entre Beck.

partie III

Explorer avant de productiser

THE DOMAIN ONTOLOGY

THE LANGUAGE YOU VERIFY IN

1 The first move in modernisation is not choosing a target architecture. It is recovering the domain, stripped of the mechanics four decades have wrapped around it.

2 What you recover is the ubiquitous language of Domain-Driven Design: the single vocabulary business and engineering both speak, with no translation layer in between.

3 It does two jobs.

1. UNDERSTANDING & DESIGN
It gives the people who never read a line of COBOL a shared, precise understanding of what the business does, and a way to design the future in business terms instead of file formats.

2. VERIFICATION
And then it does the job almost no one asks of it. It becomes the language you verify in.

4 Watch the legacy run, in production or in the oracle (recompiled to run as a behavioural reference you can replay), and you capture behaviour. Inputs in, outputs out.

5 Translate that behaviour into a domain rule a business person would recognise and sign off, and both systems can be held to it.

6 A transaction would push an account past its credit limit, and the legacy declines it.

7 Agents earn their place in two roles.

8 VERIFICATION: PROPOSE

THE ONTOLOGY IS A SUBSTRATE IN ITS OWN RIGHT.
One ontology can be realised by an infinity of target solutions. The architecture is free to change. The meaning is not. Fidelity is owed to the invariant, not to the structure.

THE ONTOLOGY (THE DOMAIN)

ACCOUNT CUSTOMER TRANSACTION
CREDIT LIMIT PAYMENT DECLINE ...

COBOL MAINFRAME COMPILED COBOL MODULAR MONOLITH MICROSERVICES EVENT-DRIVEN SYSTEM AGENTIC SYSTEM

EXECUTABLE SPECIFICATIONS (BDD)

Runs against Legacy (Oracle) **PASS**
Runs against New System **PASS**

ONE ONTOLOGY. INFINITE IMPLEMENTATIONS. ARCHITECTURE IS FREE TO CHANGE. MEANING IS NOT.

Chapitre 9

Le 3X appliqué à la modernisation

Kent Beck a observé que le logiciel vit trois phases très différentes, et que les confondre est l'une des causes les plus fréquentes de désastre. Il les a appelées Explore, Expand, Extract, les trois X. En Explore, vous cherchez ce qui marche, et la vérité inconfortable de cette phase est que la plupart de ce que vous essayez ne marche pas, donc la valeur est d'essayer à bas coût, d'apprendre vite et de jeter sans regret. En Expand, quelque chose a marché et soudain vous devez grandir pour servir une demande qui s'envole, et le problème cesse d'être quoi faire et devient tenir. En Extract, vous savez déjà ce que vous faites et vous le faites à échelle stable, donc le jeu est de presser l'efficacité, polir, optimiser. Chaque phase demande un état d'esprit différent, et appliquer l'état d'esprit de l'une à l'autre est l'erreur. Optimiser en Explore, c'est polir quelque chose que vous allez jeter. Explorer en Extract, c'est introduire du chaos là où il fallait de la stabilité.

La thèse de ce chapitre, et l'une des décisions d'attitude les plus importantes du livre, est qu'**une méthodologie de modernisation jeune est en Explore, et la traiter comme si elle était en Extract est le chemin le plus rapide vers l'échec**. Cela s'applique à deux niveaux, et il vaut mieux les séparer car il est facile de les mêler.

Le premier niveau est la méthode elle-même. Ce livre décrit une méthode qui est dans sa phase Explore. Elle n'a pas été validée contre des centaines de migrations de production. C'est une hypothèse raisonnée, un argument expliquant pourquoi elle devrait fonctionner. La traiter comme un processus mûr, la codifier en une méthodologie d'entreprise rigide et la déployer à l'échelle avant de l'avoir parcourue en petit, ce serait commettre l'erreur de Beck dans sa forme la plus pure, productiser quelque chose que vous explorez encore. La bonne attitude est l'inverse. Parcourir la méthode par cycles petits et rapides, sur des morceaux bornés, en apprenant de chaque cycle, en jetant ce qui ne marche pas, jusqu'à ce que la preuve justifie de la passer à Expand. La méthode gagne le droit de monter à l'échelle, elle ne se l'arroe pas.

Le second niveau est chaque mandat concret de modernisation, qu'il s'agisse d'un projet borné ou d'une transformation d'entreprise sur plusieurs années. Lui aussi commence en Explore, même si la pression organisationnelle pousse à feindre qu'il commence en Extract. La pression dit, nous savons déjà ce qu'il faut faire, planifions les dix-huit phases et exécutons. C'est un mensonge, et c'est un mensonge cher. Au début d'un mandat, vous n'en savez pas assez, et il vaut mieux admettre pourquoi. Vous n'êtes pas expert du métier du client. Vous êtes expert en modernisation, qui est un autre métier. Vous ne connaissez pas le domaine réel caché dans le système, vous ne savez pas où sont les frontières naturelles, vous ne savez pas quelles parties sont logiques et lesquelles mécanisme, vous ne savez pas où sont les cas rares qui vous mordront. C'est pourquoi le domaine ne se tire pas de la tête de celui qui modernise. Il se récupère de deux sources : du système, qui montre ce qu'il fait, et des experts métier, qui n'ont pas à être ingénieurs et qui mettent le nom et le pourquoi sur ce que le système ne fait que montrer faire. Feindre un plan détaillé sur cette ignorance produit un plan de fiction que la réalité détruit le premier mois.

L'alternative est de commencer par ce qui mérite d'être appelé un spike d'architecture, en empruntant le terme à la pratique agile. Un spike est une expérience bornée dans le temps dont l'objectif n'est pas de produire un livrable mais de produire de la connaissance. Vous prenez une tranche du système, fine mais complète, idéalement une qui touche les parties les plus redoutées. Fine, ça compte. Ce n'est pas une couche horizontale, ce n'est pas seulement récupérer ou seulement vérifier. C'est une aiguille qui transperce tous les étages de l'ascenseur à la fois, de la salle des machines au penthouse : récupération, reconception et vérification sur une bande étroite du domaine. Elle traverse tout, et un peu de chaque chose. Non pour la livrer, mais pour apprendre. À la fin du spike, vous savez des choses qu'aucun plan préalable

ne pouvait vous donner. Vous savez combien coûte vraiment la récupération du domaine de ce système. Vous savez si votre oracle est fidèle. Vous savez où sont les cas rares. Vous savez quelles frontières ont du sens. Avec ce savoir, le plan que vous écrivez ensuite est un vrai plan, non une fiction.

C'est ici que le bon marché agentique du chapitre cinq change les règles d'une manière profonde. Dans un monde sans agents, les spikes sont chers, parce que récupérer le domaine à la main est lent, donc la tentation de les sauter et de planifier à l'aveugle est forte, et souvent elle gagne. Dans un monde avec agents, le spike est bon marché, parce que la récupération qui coûtait des semaines coûte des jours. Et quand le spike est bon marché, vous pouvez vous permettre plusieurs, explorer plusieurs tranches, plusieurs hypothèses de frontières, plusieurs architectures candidates, avant de vous engager sur aucune. Ce qui était un luxe que presque personne ne se permettait, explorer pour de vrai avant de décider, devient l'option par défaut. L'agent n'accélère pas seulement le travail. Il rend la prudence abordable.

Il y a une discipline qui accompagne le 3X et qu'il vaut mieux énoncer sans détour. Ne productisez pas prématurément. La tentation, quand un spike réussit, est de crier victoire, de figer l'approche et de l'appliquer en masse. C'est la tentation de sauter d'Explore à Extract sans passer par Expand, de transformer une expérience réussie en un standard avant de l'avoir éprouvée sur plusieurs cas différents. Un seul spike réussi ne prouve pas que la méthode fonctionne, il prouve qu'elle a fonctionné une fois, ce qui est bien moins. Le signe que vous êtes prêt pour Expand n'est pas que quelque chose a fonctionné, c'est que vous comprenez pourquoi cela a fonctionné assez bien pour prédire quand cela ne fonctionnera pas. Tant que vous n'avez pas cela, vous êtes encore en Explore, et c'est bien d'être en Explore. C'est là qu'on apprend.

Cette attitude, parcourir par cycles rapides avant de s'engager, s'accorde avec l'ascenseur du chapitre cinq et avec la philosophie agile d'origine, la vraie, celle d'avant que le mot ne se remplisse de cérémonies : cycles courts, apprendre du monde réel, jeter sans regret ce qui ne marche pas. Chaque cycle est un voyage d'ascenseur complet, vous descendez récupérer, vous montez modéliser le domaine, vous vous arrêtez à l'étage intermédiaire décider l'architecture, vous descendez vérifier. La différence entre bien le faire et mal le faire n'est pas dans les étages, qui sont les mêmes, mais dans la vitesse et l'humilité du voyage. Rapide, pour apprendre beaucoup par unité de temps. Humble, pour jeter sans regret ce que le voyage révèle inutile. Avec cette attitude en place, le premier vrai voyage est de descendre à la salle des machines récupérer le domaine. C'est le chapitre suivant.

Chapitre 10

Récupérer le domaine

Récupérer le domaine est l'acte central de la modernisation. C'est la fondation à laquelle tout le reste est suspendu, et une fondation de travers ne se redresse pas plus haut. L'objectif est de produire un portrait fidèle du domaine que le vieux système incarne, exprimé dans les termes de la conception dirigée par le domaine, dépouillé du mécanisme, et avec chaque affirmation marquée par sa provenance. Appelons ce portrait le modèle récupéré. Ce n'est ni le vieux système ni le nouveau. C'est le domaine, le quoi, séparé du vieux comment et pas encore engagé dans aucun nouveau comment.

Il vaut mieux être très précis sur ce qu'est et ce que n'est pas ce modèle, car la confusion ici empoisonne tout le reste. Le modèle récupéré décrit l'état actuel du domaine, non l'état désiré. C'est un portrait de l'as-is, non du to-be. Cela surprend qui s'attend à ce que moderniser commence par dessiner le système idéal. Non. Cela commence par comprendre avec précision ce qu'il y a, parce que vous ne pouvez pas préserver la sémantique de quelque chose que vous n'avez pas compris, et préserver la sémantique est le contrat de fidélité du chapitre quatre. Le système idéal vient après, et il vient comme une décision de conception, marquée comme telle, non comme une part du portrait.

Le modèle récupéré a deux couches, les deux classiques du DDD. La couche tactique décrit l'intérieur de chaque partie, les entités et leurs identités, les agrégats et les invariants qu'ils protègent, les objets-valeurs, les relations. La couche stratégique décrit l'ensemble, la carte des contextes, les frontières entre les différentes parties du domaine et la façon dont elles se relient. Récupérer les deux est le travail. La tactique vous dit quelles règles vivent à l'intérieur de chaque concept. La stratégique vous dit où sont les coutures du domaine, là où plus tard vous pourrez couper.

Il y a ici une nuance que beaucoup de gens manquent et qu'il vaut mieux clouer, parce qu'elle ressemble à une contradiction et n'en est pas une. Si ce que vous récupérez est le domaine, le quoi, pourquoi le modèle récupéré contient-il aussi de la structure, qui est du comment. Parce qu'un portrait fidèle ne peut pas ignorer comment le système est organisé aujourd'hui. Le modèle récupéré est une nouvelle carte du métier, et sur cette carte vous notez deux choses distinctes, de statut distinct.

La première est la couche obligatoire : les invariants et la sémantique, l'espace du problème. C'est le quoi, il survit à toute reconception, et il est sacré. La seconde est une couche informative : la structure telle que le système tourne et s'observe aujourd'hui, les frontières qu'il a en place, quel module garde quoi, où sont les coutures. C'est de l'espace de la solution, mais non d'une conception idéale ni de celle que quelqu'un a imaginée il y a des décennies, mais du système vivant tel que l'oracle le montre. Vous la récupérez parce que c'est une information précieuse sur la façon dont le système comprend aujourd'hui son propre domaine, non parce qu'elle est sacrée. Elle est, de fait, justement ce que vous venez changer.

C'est pourquoi il n'y a pas de contradiction. Vous récupérez le domaine, et au passage vous notez la structure actuelle comme donnée, marquée comme re-décidable. Confondre les deux couches est l'erreur. Si vous traitez la structure récupérée comme sacrée, vous retournez au piège de la traduction. Si vous traitez les invariants comme négociables, vous cassez le système. La carte dessine tout, mais elle vous dit de chaque trait à quelle couche il appartient.

Le processus de récupération, dans la méthode agentique, répartit le travail selon la frontière du chapitre six. L'agent trouve. Il parcourt le système, ou plutôt, il parcourt les projections du système, les cartes qui distillent ses dimensions, et extrait des candidats, cette structure ressemble à un agrégat, cette condition ressemble à un invariant, cette donnée

ressemble à l'identité de cette entité. Chaque candidat vient avec sa provenance, je l'ai extrait du comportement, je l'ai inféré de la structure, le nom le suggère. L'architecte ratifie. Il regarde les candidats, les confronte à l'oracle quand il y a doute, les confirme ou les rejette, et surtout décide quoi faire de ce qui n'a pas de source claire.

Il y a une idée sur la ratification qui mérite d'être soulignée parce qu'elle corrige un malentendu fréquent. La ratification n'est pas là où l'incertitude se résout. C'est là où il est interdit à l'incertitude d'entrer. Cela sonne pareil mais ne l'est pas. Résoudre l'incertitude sonne comme si, dans la réunion de ratification, on discutait les doutes et on parvenait à un accord sur le douteux. Faux. Le douteux ne se ratifie pas, il se marque comme douteux et s'envoie vérifier contre l'oracle ou enquêter. Ce qui se ratifie, c'est ce qui est déjà solide, ce qui a une source ferme. La ratification est un filtre qui ne laisse passer que ce qui peut défendre sa provenance, non un tribunal qui transforme des conjectures en faits par consensus. Si vous sortez de la ratification ayant béni comme certain quelque chose qui est entré comme douteux, sans preuve nouvelle, vous avez corrompu le modèle avec l'autorité d'une signature.

Un avertissement honnête pour clore. La récupération n'est jamais complète. Il restera toujours du comportement sur des chemins qui ne s'exercent pas, des cas que l'oracle n'a pas vus parce qu'on ne les lui a pas demandés, des règles dormantes qui ne se réveillent que dans des circonstances rares. C'est du résidu irréductible, et la bonne attitude n'est pas de feindre qu'il n'existe pas mais de le marquer. Le modèle récupéré honnête inclut ses propres trous, les zones où il sait qu'il ne sait pas. Ces trous ne sont pas un échec de la récupération, ils en sont la part la plus précieuse, parce qu'ils vous disent exactement où le nouveau système peut vous surprendre, et donc où surveiller. Un modèle qui prétend être complet ment. Un modèle qui marque ses trous vous dit où regarder.

Avec le domaine récupéré, portraiture, marqué et ses trous signalés, l'ascenseur monte. Mais avant de décider la moindre architecture, il faut décider ce qui en mérite une. C'est cela, trier, et c'est le chapitre suivant.

CardDemo • étape 4/11

L'agent ne lit pas les trente et un programmes en brut. Il raisonne sur une projection : un graphe où chaque programme et chaque fichier est un nœud, et chaque accès une arête. Sur cette projection, le point chaud n'est pas à chercher, il se dessine seul. Une requête le met au jour :

```
MATCH (p:Program) -[:WRITES_FILE]->(f:File {name: 'ACCTFILE'})
RETURN p.name AS programme
ORDER BY programme
```

La réponse, ce sont quatre programmes écrivant le même fichier. Cela n'était dans aucune documentation. C'était dans le comportement, et le graphe l'a rendu visible. C'est la première preuve concrète que l'agent raisonne mieux sur la carte que sur le territoire.

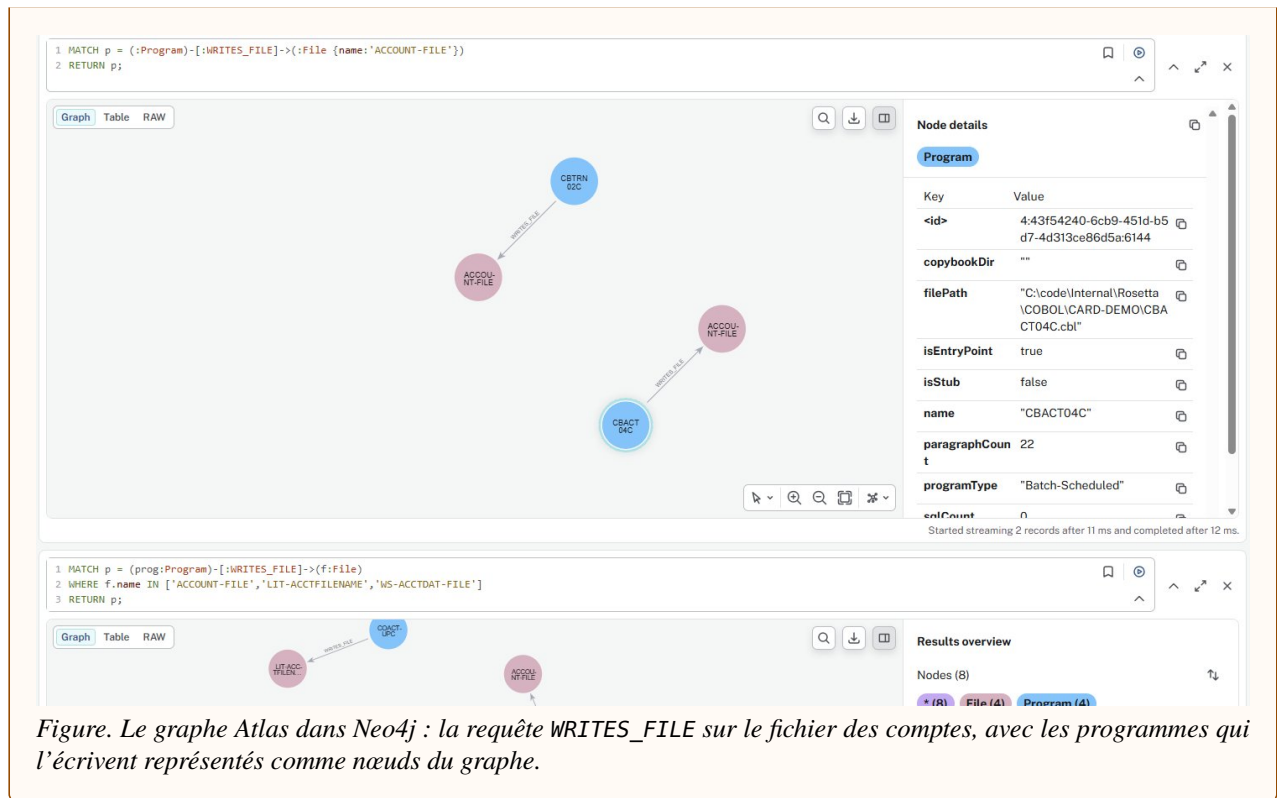


Figure. Le graphe Atlas dans Neo4j : la requête `WRITES_FILE` sur le fichier des comptes, avec les programmes qui l'écrivent représentés comme nœuds du graphe.

Chapitre 11

Récupérer, puis trier

Une fois le domaine récupéré, quand vous voyez enfin ce que vous possédez, la stratégie cesse d'être une taille unique et devient un tri, en termes simples de conception dirigée par le domaine. **Le domaine récupéré n'est pas une liste de réécriture. C'est la carte avec laquelle vous décidez quoi réécrire, quoi encapsuler et quoi mettre à la retraite.** La carte de la Figure 1 apportait déjà la réponse dessinée : la couleur est le tri.

Les sous-domaines génériques, les commodités du métier, vont à une solution du marché. Il n'y a aucune gloire à maintenir du code sur mesure pour des problèmes que le marché a résolus il y a des années, et chaque commodité que vous retirez libère des gens rares pour le travail qui compte.

Les sous-domaines de soutien, et tout code stable et simple, sont encapsulés et laissés tranquilles. Ces charges cessent de consommer vos experts COBOL chevronnés, dont on a besoin là où eux seuls servent, attester, interpréter les hotspots, valider la fidélité de la réplique. Avec la connaissance du domaine récupérée et l'oracle en place, un agent ou un développeur junior peuvent faire des changements mineurs et bien gardés en sécurité : ajouter un champ à un copybook ne devrait pas exiger un héros. Et ce qui rend ce travail sûr n'est pas la foi dans l'agent : c'est la spécification capturée au contrat du service, le filet de régression de la Partie IV appliqué à l'encapsulé.

Voici une position qu'on entend rarement dans un article sur l'IA : gardez le COBOL. Le COBOL qui est stable, correct et fait son travail en silence est un actif, non une urgence. La manœuvre mûre est de l'encapsuler derrière un contrat explicite, un service de host ouvert avec un langage publié, pour que le reste du parc consomme des capacités, non des entrailles. Si la plateforme elle-même doit bouger, relocalisez le COBOL en tant que COBOL, recompilé sur un runtime moderne et conteneurisé, derrière ce même contrat. Mieux encore, quand l'économie le permet, gardez-le là où il est et exposez-le de la même façon, par un point d'accès distant sur HTTP. Dans tous les cas le principe est identique : **envelopper pour invoquer, isoler pour ne pas hériter. Encapsuler achète de l'optionnalité ; transpiler la dépense.**

La comparaison avec la transpilation mérite d'être rendue explicite, parce que les deux se vendent comme des sorties du mainframe et ne se ressemblent en rien. Recompile conserve l'artefact que l'on maintient : le COBOL reste du COBOL, l'équipe qui le connaît reste compétente, et le compilateur garantit par construction la sémantique des données, le décimal condensé, l'encodage, l'ordre de classement, la précision, c'est-à-dire exactement là où une conversion casse en silence. Transpiler remplace l'artefact par du code généré que personne n'a écrit et que personne ne veut maintenir, le verdict du cimetière. C'est pourquoi la même technique qui construisait au chapitre trois un oracle fiable est ici une destination digne : elle préserve la sémantique par conception au lieu de la promettre.

En pratique. Un compilateur comme Raincode recompile le COBOL en IL de .NET et l'exécute sur le runtime moderne, en préservant par conception la représentation des données, le COMP-3, l'EBCDIC, l'ordre de classement et la précision décimale, de sorte que le programme qui tourne en conteneur reste, du point de vue de la maintenance, le même COBOL. Le compilateur est l'instance ; ce qui dure est le critère : pour ce que vous n'allez pas reconcevoir, choisissez la voie qui conserve l'artefact et la sémantique, non celle qui les remplace.

Cela ne contredit pas le piège de la relocalisation du début du livre ; cela le complète. Là, l'erreur était de confondre relocaliser avec moderniser le cœur, vendre un changement de plateforme comme si c'était une transformation du domaine.

Ici, pour le code que vous avez délibérément décidé de ne pas reconcevoir, relocaliser et encapsuler est exactement la bonne chose. La loi du double saut tient toujours : la relocalisation est un mouvement propre et isolé, et vous ne prétendez pas qu'elle a modernisé quoi que ce soit. La différence est dans l'honnêteté sur ce que vous achetez.

La réécriture est réservée au code qui la mérite : le cœur du métier, où le changement est constant, la rotation élevée, la complexité réelle, et où vos développeurs chevronnés passent déjà le plus clair de leur temps. Là, et seulement là, la reconception paie : réarchitecturer le cœur achète de l'agilité, réduit la dette technique et restaure la maintenabilité. Et ce cœur réécrit doit être conçu dès le premier jour pour cohabiter avec le legacy restant, avec les patrons stratégiques du DDD et ceux de déplacement de legacy, parce que pendant des années votre parc sera une fédération d'ancien et de nouveau, non un remplacement.

D'où l'inversion qui traverse tout le livre, vue depuis le portefeuille. De presque tout ce que vous récupérez, vous déciderez rationnellement de ne rien réécrire. Récupérer n'est pas le prélude obligé de la réécriture ; c'est ce qui vous permet de ne pas réécrire là où ce n'est pas le lieu, et de diriger chaque euro de reconception vers le seul endroit où il rapporte. Pour ce cœur que vous décidez bel et bien de réécrire, il est maintenant temps de décider l'architecture. Et là, enfin, l'architecte choisit. Son premier choix n'est pas un choix de code, c'est un choix de carte, et c'est le chapitre suivant. À partir d'ici, le reste du livre parle surtout de ce cœur ; l'encapsulé ne sort pas de l'histoire, il participe à la carte par son contrat et réapparaît chaque fois que le nouveau doit cohabiter avec lui.

CardDemo • étape 5/11

Élevé du graphe à la carte des contextes, voici ce que la récupération renvoie de CardDemo. Ce n'est pas un design, c'est un portrait : la structure telle que le système vivant la fait tourner aujourd'hui. Le cœur, tenue de comptes, comptabilisation des mouvements et tenue des cartes, en orange ; le soutien, où se rangent client, intérêt, consultation, facturation et rapports, en bleu ; identité et accès, le seul générique, en gris. Et en rouge le point chaud, le fichier des comptes réécrit par quatre contextes à la fois. La couleur est déjà le triage : l'orange se réécrit, le bleu s'encapsule, le gris s'achète.

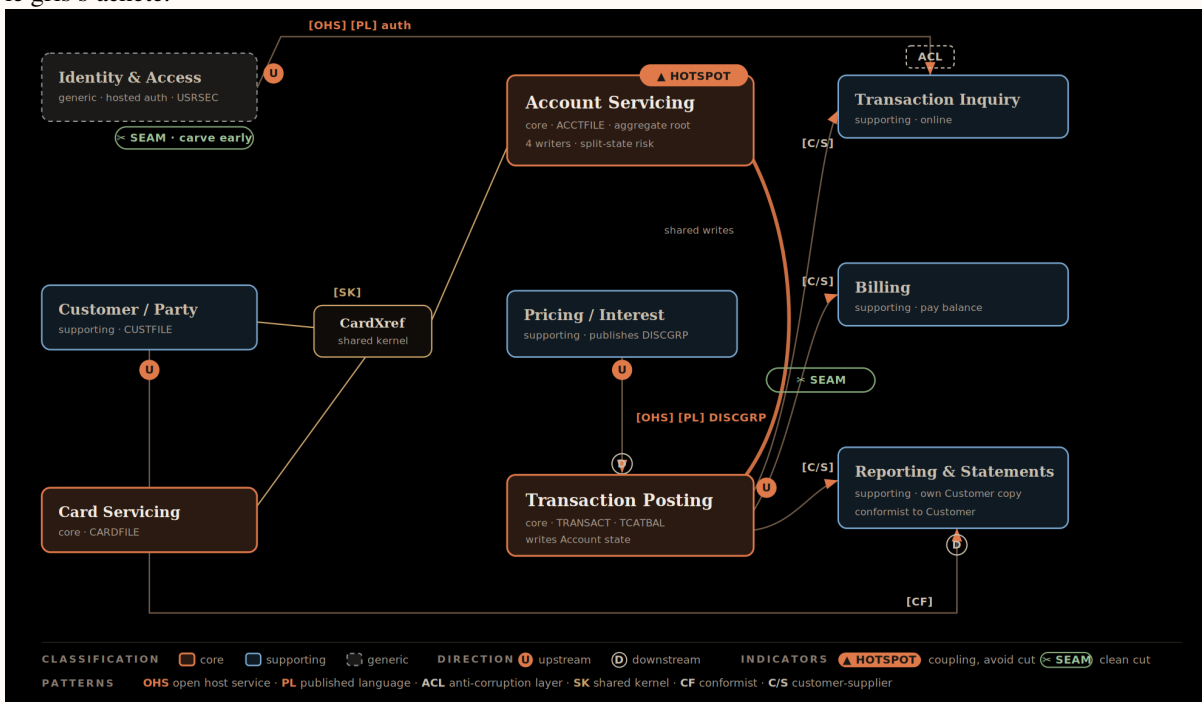


Figure. La carte des contextes as-is de CardDemo telle que la récupération la renvoie : cœur en orange, soutien en bleu, générique en gris, et le point chaud d'écriture partagée sur ACCTFILE en rouge.

Chapitre 12

La carte des contextes cible

Le premier acte de la reconception n'a pas lieu dans le code. Il a lieu sur la carte. Avant d'écrire une ligne du nouveau système, l'architecte décide la carte des contextes cible : quels contextes délimités aura le parc résultant, quel modèle et quel langage vivent dans chacun, et comment ils se relient entre eux. **Le premier choix d'architecture n'est pas un choix de code, c'est un choix de carte.** C'est l'acte que la traduction saute entièrement, et c'est là que la modernisation se gagne ou se perd comme conception.

Souvenez-vous que vous arrivez ici avec deux cartes en main. Le modèle récupéré apporte la carte as-is, la structure telle que le système vivant la montre, notée comme donnée re-décidable. La carte cible est autre chose : c'est une décision. Sur un même domaine récupéré tiennent beaucoup de cartes possibles, et en choisir une est un jugement de l'architecte, non une trouvaille de l'agent. Chaque frontière que la carte cible conserve de l'as-is est une décision de conserver. Chaque frontière qu'elle corrige est une décision de corriger. Les deux se marquent comme décisions, avec la discipline de provenance habituelle, parce que la carte cible ne se récupère nulle part : elle se choisit.

Il vaut la peine de préciser ce qui se décide en traçant un contexte, parce que c'est plus qu'un cadre avec un nom. Un contexte délimité est la frontière à l'intérieur de laquelle un modèle du domaine et son langage sont cohérents. Et le langage omniprésent, disons-le sans ambiguïté, n'est pas un pour tout le métier : il y en a un par contexte. Dans CardDemo, « compte » ne signifie pas exactement la même chose pour le servicing qui le gouverne que pour les rapports qui l'agrègent, et cette différence n'est pas un défaut à unifier, c'est une information à protéger par une frontière. Forcer un langage unique sur des significations distinctes produit le pire des modèles, celui qui paraît cohérent et ne l'est pas. La carte existe pour que chaque signification ait sa maison.

La carte inclut tous les contextes, pas seulement ceux que vous allez réécrire. Ici le tri du chapitre précédent devient cartographie. Le COBOL encapsulé n'est pas un résidu hors du plan : c'est un contexte délimité de plein droit, avec le modèle qu'il a, son contrat et ses relations dessinées. Une carte qui ne dépeint que le neuf est de la propagande. La carte honnête dépeint la fédération entière d'ancien et de nouveau que vous allez opérer pendant des années, parce que les relations qui demandent le plus de soin sont justement celles qui traversent cette couture.

Les relations entre contextes ont des noms, et les employer discipline la conception. Le contexte qui offre des capacités à de nombreux consommateurs publie un service de host ouvert avec un langage publié. Le contexte qui consomme et veut protéger son modèle dresse une couche anticorruption. Ce ne sont pas des alternatives entre lesquelles choisir : ce sont les deux extrémités de la même relation, le host ouvert et le langage publié vivent du côté de celui qui offre, la couche anticorruption du côté de celui qui consomme, et une relation saine peut avoir les deux. Entre contextes dont les équipes négocient leurs priorités, il y a une relation de client et fournisseur. Là où vous acceptez le modèle de l'autre tel quel, sans traduire, vous êtes conformiste, ce qui est bon marché et honnête quand le modèle d'autrui est bon. Et là où intégrer coûte plus que cela ne vaut, la bonne relation est chemins séparés : dupliquer un peu et ne pas intégrer est aussi une relation, la moins chère de toutes quand elle s'applique.

CardDemo laisse voir la carte cible en petit. Le cœur, le servicing des comptes et le posting des mouvements, se réécrit, et le hotspot du fichier des comptes se résout sur la carte avant de se résoudre dans le code : l'état du compte passe à un seul contexte propriétaire de son écriture, et les autres le consomment par contrat. La sécurité, stable et isolée, reste comme contexte encapsulé derrière un host ouvert. Les rapports consomment en aval, conformistes si le langage publié

leur suffit, avec leur propre couche anticorruption s'ils ont besoin de leur propre modèle. Trois types de relation, un seul domaine, et aucune de ces flèches n'était dictée : toutes sont des choix que la carte laisse écrits.

La carte a en outre un prix que l'agent peut mettre devant chaque option. Chaque frontière qu'elle redessine par rapport à l'as-is diverge de l'oracle et renchérit la vérification ultérieure ; chaque frontière qu'elle conserve la rend moins chère. Une carte ambitieuse paie son ambition dans la Partie IV. Ce n'est pas une raison pour éviter l'ambition, c'est une raison pour être ambitieux avec le prix sous les yeux, et pour se rappeler que la carte peut évoluer ensuite, le filet en place, au lieu de naître parfaite.

C'est pourquoi l'ordre prudent fixe la carte d'abord et gèle les contrats au plus tôt. Les contrats entre contextes sont la chose la plus chère à changer ensuite, parce que les deux côtés de chaque couture y sont suspendus. Carte fixée, contrats publiés, et alors seulement l'ascenseur descend à l'intérieur de chaque contexte qui se réécrit. Cet intérieur est la conception tactique, et c'est le chapitre suivant.

CardDemo • étape 6/11

Dans CardDemo, « compte » ne signifie pas la même chose pour la tenue de comptes qui le gouverne que pour les rapports qui l'agrègent. Pour la tenue de comptes c'est la ligne de crédit vivante, qui protège son invariant ; pour les rapports c'est une ligne qu'on additionne. Le même nom, deux modèles :

```
// Contexte Tenue de comptes : le compte protège sa règle de cycle
public sealed class Compte
{
    public Money CreditDisponibleDuCycle() =>
        plafond - (creditCycle - debitCycle);
}
```

```
// Contexte Rapports : le compte est une ligne qu'on agrège
public sealed record LigneCompte(long CompteId, decimal Solde);
```

La carte objectif leur donne des contextes distincts au lieu d'un « compte » unique qui ment aux deux.

CardDemo • étape 7/11

CardDemo laisse voir la carte objectif en petit. Le cœur, tenue des comptes et comptabilisation des mouvements, est réécrit, et le point chaud du fichier des comptes se résout sur la carte avant de se résoudre dans le code : l'état du compte vient à n'avoir qu'un seul contexte propriétaire de son écriture, et les autres le consomment par contrat. L'intérêt, stable, reste encapsulé derrière un hôte ouvert avec son langage publié. L'identité et l'accès, génériques, sont découpés tôt et remplacés par un service hébergé, consommé derrière une couche anticorruption. Les rapports consomment en aval avec leur propre couche anticorruption. Trois mouvements de triage et trois types de relation, un seul domaine, et aucune de ces flèches n'était dictée.

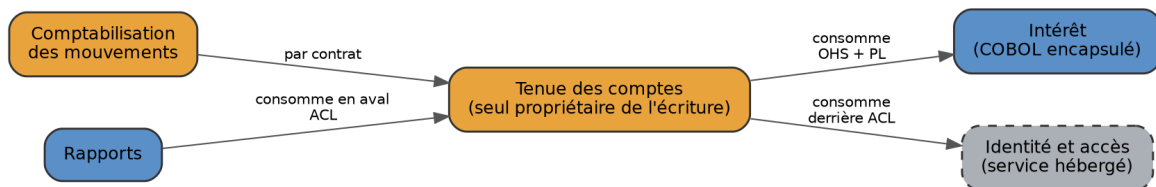


Figure. La carte des contextes objectif de CardDemo : le point chaud résolu dans un unique contexte propriétaire de l'écriture du compte, le reste consommant par contrat (OHS+PL côté fournisseur, ACL côté consommateur).

Chapitre 13

La conception tactique du cœur

La carte fixée et les contrats publiés, l'ascenseur descend d'un étage et entre dans chaque contexte du cœur. Vous avez un modèle du domaine, fidèle, marqué, avec la sémantique séparée de la vieille structure, et une carte qui dit quels contextes se réécrivent. Maintenant vous décidez comment sera chacun à l'intérieur. C'est de la conception, au sens plein, et donc c'est le territoire de l'architecte, non de l'agent. L'agent dresse la table d'options, comme nous l'avons vu. L'architecte choisit le plat.

La première idée libératrice est celle que nous avons déjà semée au chapitre quatre. **Sur un même domaine récupéré tiennent bien des architectures.** Le domaine est unique. Les manières de le structurer sont nombreuses. La carte a déjà regroupé les contextes ; à l'intérieur de chacun vous pouvez choisir un style d'architecture ou un autre, le paradigme, procédural, orienté objet ou fonctionnel, et décider quelles parties méritent une richesse de modèle et lesquelles peuvent rester simples. Aucune de ces décisions n'est dictée par le domaine. Ce sont toutes des jugements de l'architecte sur ce qui sert le mieux les contraintes d'aujourd'hui, maintenabilité, performance, autonomie des équipes, ce qui compte dans votre cas. Cette multiplicité est exactement le prix d'avoir séparé la sémantique de la structure. Si vous aviez conservé la vieille structure, il n'y aurait pas de choix. Comme vous l'avez lâchée, il y a un éventail.

Le travail de conception se divise en deux moitiés de nature très différente, et les séparer évite bien des confusions. Il y a une moitié mécanique et une moitié de choix.

La moitié mécanique est la traduction de patrons du domaine récupéré en patrons de l'architecture cible, quand cette traduction est directe. Une opération du domaine devient une unité d'exécution de la nouvelle architecture. Un événement du domaine devient un message. Un traitement par lots devient un flux. Ces correspondances sont assez mécaniques une fois décidé le style d'architecture, et ce sont justement le genre de travail que l'agent fait bien et vite. Ici les frameworks concrets entrent comme exemples. Un framework qui organise le code en tranches verticales, chacune autour d'une opération métier et non d'une couche technique, est un exemple d'architecture cible qui s'accorde naturellement avec un domaine récupéré en termes d'opérations et de règles, parce que la tranche verticale préserve la cohésion de chaque opération. Mais le framework est l'exemple. Le concept durable, c'est qu'il y a une part de la traduction vers la nouvelle architecture qui est mécanique et délégable.

En pratique. Dans l'écosystème .NET, un framework d'architecture en tranches verticales comme Wolverine organise chaque opération métier dans sa propre tranche, avec son message et son gestionnaire, au lieu de la répartir entre couches techniques. L'opération de CardDemo qui autorise un mouvement et met à jour le solde, aujourd'hui répartie entre des programmes d'imputation qui partagent le fichier des comptes, se traduit alors en une tranche avec son message d'entrée et son gestionnaire ; un événement du domaine, en un message ; le calcul d'intérêt par lots, en un flux. D'autres styles, comme l'hexagonal, arrivent au même endroit par un autre chemin. Le framework est l'instance ; ce qui est durable, c'est que la tranche suit l'opération métier, non la couche technique.

La moitié de choix est là où vit l'architecte. Ici sont les décisions que le domaine ne dicte pas. Transformez-vous un concept que le vieux système tenait anémique, un sac de données sans comportement, en un agrégat riche qui protège ses propres invariants ? Redessinez-vous une frontière que le vieux système avait mal posée ? Transformez-vous une donnée isolée en un objet-valeur doté de son propre sens, comme le solde du compte, aujourd'hui un champ numérique

dans le fichier, en un montant qui porte toujours sa devise et refuse de s'additionner avec une autre ? Chacune de ces décisions est un choix de conception, et chacune a des conséquences, non seulement sur la qualité du nouveau système mais, crucialement, sur le coût de la vérification ultérieure. Une architecture plus fidèle à la vieille structure est moins chère à vérifier, parce qu'elle ressemble plus à l'oracle. Une architecture plus ambitieuse, qui redessine des frontières, est plus chère à vérifier, parce qu'elle diverge davantage. L'agent peut et doit quantifier cette conséquence pour chaque option. L'architecte décide avec cette information sur la table.

Il y a une décision récurrente qui illustre bien pourquoi le jugement humain est irremplaçable ici, et c'est la relocalisation d'un invariant. Dans le vieux système, une règle pouvait être appliquée en plusieurs endroits à la fois, répétée, parce que plusieurs parties du système écrivaient la même donnée et que chacune vérifiait la règle de son côté. En reconcevant, le naturel est de relocaliser cette règle en un seul endroit, l'agrégat qui est propriétaire de la donnée, pour qu'il la protège une fois et bien. Mécanique, en apparence. Mais il y a un piège que seul le jugement détecte. Relocaliser la règle en un seul site suppose que toutes les vieilles copies de la règle étaient identiques. Et si elles ne l'étaient pas ? Et si l'une des parties qui écrivait la donnée appliquait la règle et une autre non, délibérément, parce que dans ce flux précis la règle ne devait pas s'appliquer ? Alors consolider les copies en une seule n'est pas un nettoyage, c'est un changement de comportement, et possiblement une grave erreur. Détecter cette divergence exige de regarder le domaine récupéré d'un œil critique, et de décider si les copies étaient la même règle mal répétée ou des règles distinctes qui se ressemblaient. Cela, c'est du jugement. L'agent peut vous signaler qu'il y a plusieurs copies et vous montrer si elles diffèrent. Décider ce que cette différence signifie et quoi en faire est de l'architecte, et c'est exactement le genre de décision qui, si vous la déléguez, vous explose des mois plus tard.

Il y a un ordre prudent pour tout cela, et il se relie de nouveau au 3X. Ne poursuivez pas le modèle ambitieux au premier essai. La couche stratégique a déjà été résolue au chapitre précédent : la carte fixée, le vieux système enveloppé dans sa couche anticorruption, les contrats publiés. À l'intérieur, commencez conservateur. Chaque contexte reste mince, presque un calque du comportement que montre l'oracle, même anémique et peu élégant. Cette conception est bon marché à vérifier, parce qu'elle ressemble à l'oracle, donc vous fermez l'équivalence vite et vous avez un nouveau système qui fait démonstrablement la même chose que le vieux. Alors seulement, le filet de sécurité en place, vous abordez les améliorations ambitieuses, enrichir le modèle tactique de chaque contexte et, s'il le faut, remonter à la carte pour redessiner une frontière, chaque pas avec sa propre vérification.

La spécification du domaine, que nous verrons dans la partie suivante, sert de gate aux deux conceptions sans changer une virgule. Peu importe que vous vérifiez la conservatrice ou l'ambitieuse : comme elle est écrite en termes du domaine et non d'une architecture, elle mesure les deux avec la même aune. C'est pourquoi vous pouvez commencer conservateur, fermer l'équivalence, et enrichir ensuite sans réécrire la spécification, parce que ce qu'elle mesure, c'est le quoi, qui ne change pas, non le comment, qui change. D'abord démontrez que vous ne cassez rien avec une conception humble. Ensuite améliorez, pas à pas, sans jamais lâcher le filet. Optimiser avant d'avoir le filet, c'est, encore une fois, explorer là où il fallait stabiliser.

Cela nous mène droit au pilier qui rend tout ce qui précède sûr au lieu de téméraire. Vous avez récupéré un domaine et décidé une nouvelle architecture qui diverge de la vieille. Comment démontrez-vous, non supposez, démontrez, que le nouveau système signifie la même chose que le vieux ? C'est la Partie IV, et c'est là que la méthode cesse d'être une bonne intention et devient un engagement vérifiable.

CardDemo • étape 8/11

L'opération de CardDemo qui autorise un mouvement est aujourd'hui répartie entre la comptabilisation et le REWRITE du fichier. Dans le nouveau design elle devient une seule tranche verticale, un message et son gestionnaire :

```
public record AutoriserMouvement(long CompteId, Money Montant);

public async Task<Resultat> Handle(AutoriserMouvement c)
{
    var compte = await comptes.Charger(c.CompteId);
    return compte.Autoriser(c.Montant); // applique les règles du cycle
}
```

Le calcul d'intérêt par lots, qui parcourt les comptes à la clôture du cycle, devient un flux à part. Chaque tranche

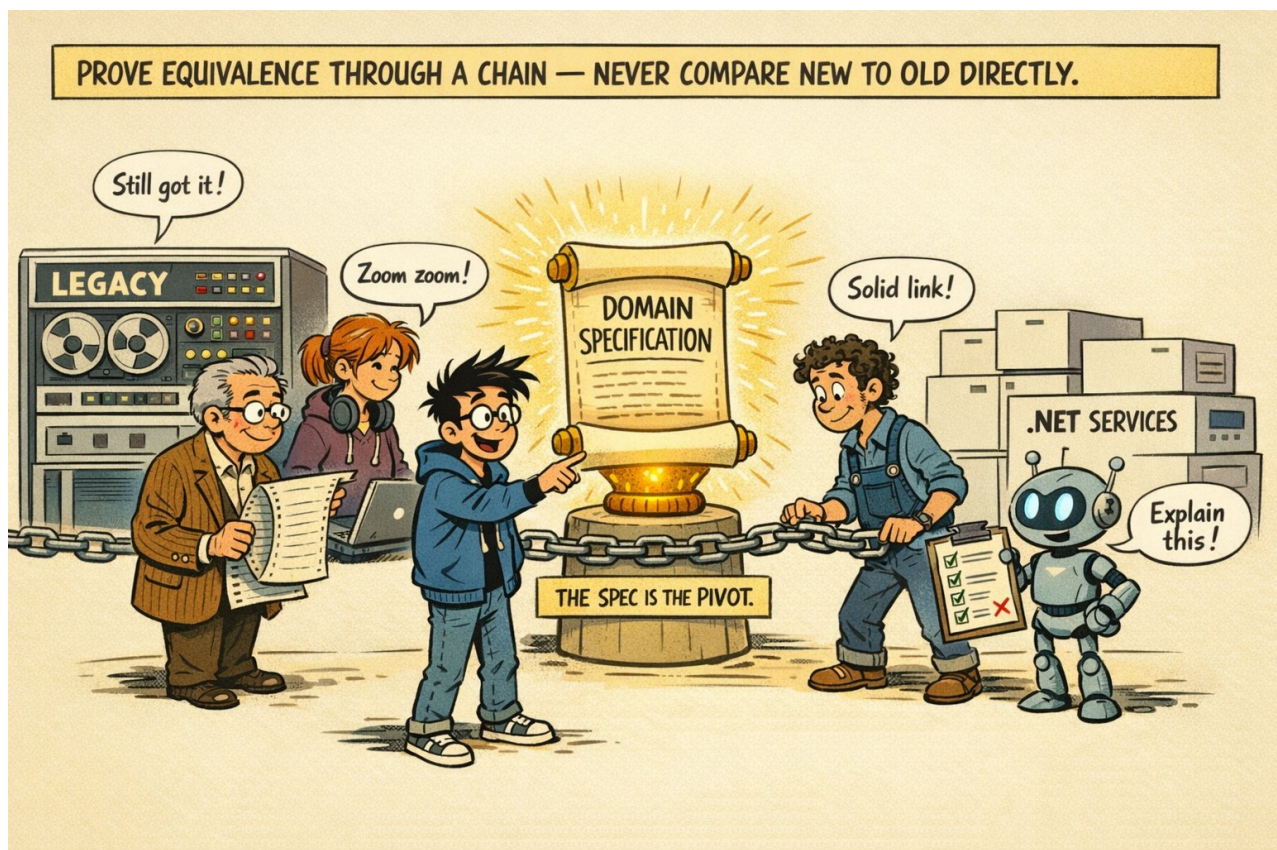
suit une opération métier, non une couche technique.

partie IV

Prouver que vous ne l'avez pas cassé

Chapitre 14

La chaîne transitive



Voici le problème dans sa forme la plus crue. Vous avez le vieux système, qui fonctionne mais dont vous abhorrez la conception. Vous voulez un nouveau système, avec une conception que vous avez choisie et qui diverge exprès de la vieille. Et vous devez démontrer que le nouveau se comporte comme le vieux dans tout ce qui compte. La tentation est évidente. Comparer le nouveau avec le vieux, directement. Même entrée aux deux, comparez les sorties, si elles coïncident, équivalents.

Cette comparaison directe est plus difficile qu'il n'y paraît, et parfois impossible à bien faire, pour une raison qui vient du chapitre quatre. Le nouveau système et le vieux divergent à l'intérieur exprès. Ils ont des frontières différentes, des structures de données différentes, ils parlent peut-être même des langues différentes à leurs interfaces, parce que le vieux parle en termes de son vieux mécanisme et le nouveau en termes de son domaine propre. Dans CardDemo, le vieux répond avec un code retour et une COMMAREA positionnelle là où le nouveau répond « mouvement autorisé » avec un objet de domaine : ils disent la même chose en deux langues qui ne se ressemblent pas. Comparer deux systèmes qui

ne présentent même pas la même forme d'entrée et de sortie exige une couche de traduction entre eux, et cette couche de traduction est du nouveau code, non vérifié, qui s'interpose juste là où vous voulez de la certitude. Vous finissez par confier votre preuve d'équivalence à un traducteur dont personne n'a démontré la correction. La comparaison directe met une inconnue au centre de la preuve.

La solution est un détour qui se révèle plus droit que la ligne directe. **Ne comparez jamais le nouveau système contre le vieux en direct. Construisez une chaîne d'équivalences transitive.** L'idée est d'introduire des pas intermédiaires choisis de sorte que chaque maillon de la chaîne soit facile à vérifier, et de laisser la transitivité faire le reste. Si A est équivalent à B, et B équivalent à C, alors A est équivalent à C, sans avoir jamais comparé A à C directement.

Voyons la chaîne. Le premier maillon est de construire une réplique du vieux système qui préserve sa structure, non sa conception, sa structure. C'est exactement l'oracle du chapitre trois, la copie qui tourne vite et en local. Comme elle préserve la structure du vieux, démontrer que cette réplique est équivalente au vieux système original est facile, parce qu'ils sont presque identiques à l'intérieur, donc la comparaison est presque une identité et peut se faire avec une vérification fine, comparant même les détails internes, parce que les structures se correspondent. Ce maillon est bon marché précisément parce qu'il n'y a pas de divergence de conception à franchir. Une fois fermé, vous avez une réplique fidèle du vieux comportement qui vit sur votre machine et répond en millisecondes. Et, point important, à partir d'ici le vieux système original peut se retirer de l'équation du travail quotidien, parce que son comportement vit déjà, fidèlement, dans la réplique.

Le deuxième maillon ne compare pas des systèmes, il capture du comportement. Vous utilisez la réplique fidèle comme oracle et vous observez, systématiquement, ce qu'elle fait face à un ensemble bien choisi d'entrées. Le résultat de cette observation, vous l'écrivez comme une spécification exécutable, une collection de cas qui disent, face à cette situation et cette requête, le comportement observable est celui-ci. Cette spécification est le maillon suivant de la chaîne, et elle a une propriété magique. Elle est exprimée en termes du domaine, non en termes de la structure d'aucun système. Elle ne dit ni ce que fait le vieux système à l'intérieur ni ce que fera le nouveau à l'intérieur. Elle dit ce qui s'observe, en langage de domaine. C'est pourquoi elle peut servir de patron aux deux.

Le troisième maillon est le nouveau système. Vous le construisez, avec sa conception divergente, et vous démontrez qu'il satisfait la spécification capturée. Vous ne le comparez pas au vieux. Vous le comparez à la spécification, qui est en langage de domaine et n'exige donc pas une couche de traduction douteuse, elle exige seulement que le nouveau système, exprimé dans son propre domaine propre, produise les observables que la spécification dicte.

Et là la transitivité se ferme. Le vieux système est équivalent à sa réplique fidèle, maillon un, vérification fine. La réplique fidèle produit la spécification, maillon deux, par observation directe. Le nouveau système satisfait la spécification, maillon trois. Donc le nouveau système est équivalent au vieux, sans les avoir jamais comparés directement, sans avoir jamais construit cette couche de traduction douteuse entre deux systèmes qui divergent à l'intérieur. La spécification, en langage de domaine, est le pivot qui connecte les deux mondes sans les toucher l'un à l'autre.

Il y a une élégance supplémentaire dans cette chaîne qu'il vaut la peine de noter. La granularité de la vérification suit la ressemblance structurelle de chaque maillon, et c'est un principe général qui mérite d'être retenu. Là où deux choses se ressemblent beaucoup à l'intérieur, maillon un, vieux contre réplique, vous pouvez et devez vérifier fin, en regardant les détails internes, parce que la correspondance existe. Là où deux choses divergent à l'intérieur exprès, maillon trois, spécification contre nouveau système, vous ne pouvez pas regarder les détails internes parce qu'ils ne se correspondent pas, donc vous ne vérifiez que l'observable depuis l'extérieur, et l'observable se nomme dans le langage du domaine, dans l'espace du problème, non en termes techniques d'aucune solution. La granularité de la preuve s'ajuste à combien se ressemblent les choses que vous comparez. Forcer une comparaison fine entre des choses qui divergent est l'erreur que la chaîne transitive évite.

La chaîne réconcilie en outre le tri avec la vérification, et l'image est nette : **le tri décide combien de chaîne vous parcourez.** Pour le COBOL que vous avez décidé d'encapsuler et de relocaliser, le premier maillon est toute la vérification dont il a besoin : la fidélité de la réplique recompilée démontrée, ce contexte est déjà là où il devait être, derrière son contrat. Seul le cœur que vous réécrivez parcourt la chaîne entière, parce que lui seul diverge par conception. La chaîne n'est pas un péage uniforme ; c'est une route dont chaque charge parcourt le tronçon que sa destination exige.

Il reste à développer ce qu'est exactement cette spécification capturée, comment elle s'écrit pour être à la fois patron de preuve, filet de régression et documentation, et pourquoi son indépendance vis-à-vis de la langue des deux systèmes est ce qui protège votre liberté de reconception. C'est le chapitre suivant.

CardDemo • étape 9/11

Le vieux CardDemo parle par une COMMAREA positionnelle, le copybook C0COM01Y, où chaque donnée vit à sa place par décalage :

```
01 CARDDEMO-COMMAREA.
```

```
05 CDEM0-ACCT-ID      PIC 9(11).
```

```
05 CDEM0-ACCT-STATUS  PIC X(01).
```

Le nouveau parle par un objet de domaine :

```
public record MouvementAutorise(long CompteId, Money CreditRestant);
```

Ils disent la même chose en deux langages qui ne se ressemblent pas. Les comparer en direct exigerait un traducteur entre les deux, du code non vérifié placé juste là où vous voulez la certitude. C'est pourquoi la chaîne ne les compare pas l'un à l'autre : elle les relie à travers la spécification, écrite en langage de domaine, qui sert de patron aux deux.

Chapitre 15

Le comportement comme spécification exécutable

La spécification capturée est le pivot de toute la vérification, elle mérite donc un chapitre entier sur ce qu'elle est et pourquoi elle a la forme qu'elle a. L'idée centrale est que le comportement observé de l'oracle s'écrit comme une spécification exécutable, une collection de cas qu'une machine peut exécuter et vérifier, non un document en prose que quelqu'un doit interpréter.

La forme la plus utile que je connaisse pour cela est l'exemple tabulé, dans le style étant donné ceci, quand cela, alors ceci. Chaque cas décrit un état de départ, étant donné ceci, une action, quand cela, et l'observable attendu, alors ceci. Une ligne du tableau est un cas. Le tableau entier est la spécification d'une opération du domaine. Cette forme a une énorme vertu pédagogique, parce qu'elle se lit presque comme de la langue naturelle et qu'un expert métier qui ne programme pas peut la relire, et en même temps elle est exécutable, parce que chaque ligne peut se lancer contre un système et vérifier si l'observable réel coïncide avec l'attendu. Cette idée n'est pas neuve. Ward Cunningham l'a introduite en 2002 avec FIT, le Framework for Integrated Test : des tableaux qu'un expert métier peut lire et qu'une machine en même temps exécute.

Le plus important dans la spécification, et ce qui en fait le pivot de la chaîne transitive, c'est la langue dans laquelle elle est écrite. **La spécification s'écrit dans le langage omniprésent du domaine, non dans la langue de l'un ou l'autre des deux systèmes.** Le vieux système parle en termes de son mécanisme, ses structures de mémoire, ses codes techniques. Le nouveau système parle en termes de son domaine propre. Si vous écriviez la spécification dans la langue du vieux, vous attacheriez le nouveau système à la vieille forme et perdriez la liberté de reconception qui vous a tant coûté à gagner. Si vous l'écriviez dans la langue du nouveau, vous ne pourriez pas l'utiliser pour vérifier contre le vieux. La seule forme qui serve aux deux est une troisième langue, celle du domaine pur, indépendante des deux implémentations. C'est la langue dans laquelle l'expert métier décrirait la règle sans rien connaître aux ordinateurs. Avec une précision que le chapitre douze a laissée établie : le langage omniprésent est un par contexte, non un pour tout le métier. Chaque contexte se spécifie dans son propre langage, et les cas qui traversent les contextes parlent le langage publié du contrat qui les unit.

Cela a une conséquence architecturale concrète. Comme la spécification est en langage de domaine et que chaque système parle sa propre langue, il faut une pièce de traduction par système, qui prend un cas en langage de domaine et l'exécute contre ce système concret, en traduisant l'entrée dans la langue du système et la sortie de retour vers le domaine. Il y a une de ces pièces pour le vieux et une autre pour le nouveau. La vertu d'isoler la traduction dans ces pièces est que la dépendance à la langue de chaque système reste confinée là, dans un endroit petit et bien borné, si borné qu'on peut le tester unitairement et à fond, en épuisant ses cas, au lieu de contaminer la spécification. La spécification reste pure, en domaine, réutilisable, et la saleté de chaque langue concrète vit isolée dans sa pièce de traduction. Cette pureté est ce qui protège votre liberté de reconception, parce que vous pouvez changer entièrement le nouveau système, sa langue comprise, sa pile technologique entière, et la spécification tient encore, vous ne changez que sa pièce de traduction.

La spécification capture l'observable complet du domaine, et il faut être prudent sur ce que veut dire complet. Cela veut dire non seulement la réponse directe à la requête, mais aussi les effets de bord qui importent au domaine, le changement d'état que l'opération laisse derrière elle. Si une opération, en plus de répondre, modifie l'état d'une entité, ce changement d'état fait partie de l'observable et doit figurer dans la spécification. Mais, et c'est la clé, ces effets de bord s'observent

à travers des requêtes de domaine, en demandant au système son état en termes de domaine, non en espionnant son magasin de données de l'intérieur. Espionner le magasin briserait la boîte noire et rattacherait la preuve à la structure de stockage concrète, interdisant la reconception. On demande l'état en langage de domaine, comme on a demandé la réponse. Dans CardDemo, si autoriser un mouvement réduit le crédit disponible, vous ne lisez pas le fichier des comptes : vous demandez au système le solde du compte en langage de domaine et vérifiez qu'il a baissé de ce qu'il devait.

Il y a une limite honnête qu'il vaut mieux déclarer. Complet veut dire tout l'observable de domaine qui importe, non littéralement tout effet. Les effets purement techniques, une entrée dans un journal, un compteur interne, un code d'état de la plomberie, ne se capturent pas, parce qu'ils ne sont pas du domaine, ils sont du mécanisme, et les comparer serait comparer précisément ce que vous avez le droit de changer. Et il y a une limite plus profonde, vous ne pouvez capturer que ce que le système expose. S'il y a un effet de domaine qu'aucune requête ne met en lumière, il reste hors de la spécification, c'est un angle mort. Capturer tout l'observable est l'objectif, mais cela exige que le système expose, via des requêtes de domaine, tous ses effets de domaine, et cela ne vient pas gratuitement, il faut le concevoir.

Pour finir, une observation sur pourquoi cette spécification est si rentable. Chaque ligne du tableau vit trois vies à la fois. Tant que le vieux système ou sa réplique restent vivants, la ligne est un cas de test de parité, elle vérifie que le vieux et le nouveau coïncident. Quand le vieux part à la retraite, la même ligne devient un test de régression du nouveau système, elle vérifie qu'aucun changement futur ne casse le comportement hérité. Et toujours, dès le premier jour, la ligne est de la documentation vivante du domaine, dans une langue que le métier comprend, et que les agents comprennent aussi et peuvent restituer sous bien des formes, en tableaux, en markdown, en diagrammes. Elle ne ment pas, parce qu'elle est exécutable et se casserait si elle mentait. Peu de choses en ingénierie donnent trois usages pour le prix d'un. La spécification de domaine en est une, et c'est pourquoi l'effort de bien la capturer en vaut la peine.

Une dernière pièce manque pour que la vérification soit solide, et c'est la plus subtile de toutes. Ce que vous comparez exactement quand vous comparez l'observable, et dans quel ordre vous faites les choses. C'est le dernier chapitre de cette partie.

CardDemo • étape 10/11

Dans CardDemo, autoriser un mouvement diminue le crédit disponible. Pour vérifier l'effet on ne lit pas le fichier des comptes : on demande au système le solde en langage de domaine et on vérifie qu'il a baissé de ce qu'il devait. Lire le magasin attacherait la preuve à la structure de stockage ; demander le domaine la laisse libre.

```
// casse la boîte noire : attache la preuve au stockage
var solde = LireFichierComptes(compteId).ChampSolde;

// observable de domaine : demande en langage de domaine
var solde = await comptes.ObtenirCreditDisponible(compteId);
```

Chapitre 16

Parité boîte noire et la loi d'ordre

Nous savons déjà que nous comparons l'observable depuis l'extérieur, non les entrailles. Cette façon de vérifier, nous l'appelons parité boîte noire, parce qu'elle traite chaque système comme une boîte fermée dont on ne voit que les entrées et les sorties. Mais boîte noire ne veut pas dire comparer à l'aveugle tout ce qui sort. Cela veut dire choisir avec soin ce qui, dans ce qui sort, compte comme observable de domaine et ce qui est du bruit technique à ignorer. Ce choix, ce qu'on compare, est le cadran le plus délicat de toute la vérification, et le mal calibrer ruine la preuve dans l'une de deux directions opposées.

Si vous comparez de trop, en incluant dans la comparaison des détails techniques que le nouveau système a le droit de faire autrement, la preuve échoue constamment sur des différences qui ne comptent pas, un code interne différent, un format de date différent, un ordre de champs différent. Vous vous noyez dans les faux positifs, la preuve devient du bruit, et vous finissez par l'ignorer, ce qui est le pire qui puisse arriver à un test. Si vous comparez de trop peu, en excluant des choses qui étaient bel et bien des observables de domaine, la preuve passe même si le comportement a vraiment changé, et elle vous donne une fausse sécurité, ce qui est encore pire, parce qu'elle vous laisse casser le système la conscience tranquille. Entre comparer de trop et de trop peu, il y a une calibration correcte, et la trouver est le métier.

La règle de calibration est celle-ci. Classez chaque champ de l'observable dans l'une de plusieurs catégories. Les résultats de domaine et l'état de domaine résultant sont en comparaison stricte, ils doivent coïncider exactement, jusqu'au dernier centime du solde du compte après la clôture du cycle, parce que ce sont la sémantique que vous avez promis de préserver. Les champs purement techniques, du mécanisme sans conséquence de domaine, s'excluent, parce qu'ils sont justement ce que vous avez le droit de changer. Les champs non déterministes, un identifiant généré, un horodatage, se normalisent avant comparaison, on les réduit à une forme canonique qui ignore ce qui varie légitimement. Et il y a une catégorie dangereuse, les champs ambigus ou entremêlés, où une valeur technique cache une conséquence de domaine, et ceux-là, il faut les démêler avec soin, en comparant la conséquence de domaine et non la valeur technique qui la transporte. Et attention à ce qui paraît technique et ne l'est pas. L'ordre des champs du relevé que les Rapports consomment, ou le format du groupe de tarification qui gouverne la recherche du taux, paraissent du mécanisme, mais si quelqu'un en aval en dépend ce sont des règles de domaine, et elles vont en comparaison stricte.

La règle d'or de cette classification est de comparer strict par défaut et d'exclure seulement sur preuve. La charge de la preuve repose sur l'exclusion, non sur l'inclusion. Si vous doutez qu'un champ importe, vous le comparez, parce que le risque de comparer de trop est un bruit gênant, tandis que le risque d'exclure de trop est une cécité dangereuse. Vous n'excluez un champ que lorsque vous pouvez démontrer qu'il est technique et qu'il ne traîne aucune conséquence de domaine. Et cette démonstration devrait pouvoir se tracer, chaque champ exclu trace à une pièce de mécanisme concrète sans effet de domaine. Exclure par confort, parce que le champ gêne, c'est ouvrir la porte à la cécité.

Le cas des erreurs illustre bien la subtilité. Quand une opération échoue, comparez-vous le mode d'échec ou le dénouement de domaine ? La bonne réponse est presque toujours le dénouement de domaine. Si le vieux système rejette une opération par une règle métier, et que le nouveau la rejette par la même règle métier, ils sont équivalents, même si à l'intérieur le mécanisme du rejet est complètement différent. Ce qui importe, c'est que l'opération a été rejetée pour cette raison de domaine, non comment le rejet a été implémenté. En revanche, si le vieux échoue par une panne technique, ce n'est pas du domaine, c'est du mécanisme, et cela ne devrait pas faire partie de la comparaison. Distinguer le rejet

métier de la panne technique est exactement le démêlage du chapitre quatre, où nous avertissions que là se cache la perte silencieuse de sémantique. La parité bien calibrée est là où cette distinction devient opérationnelle.

Il manque le second thème du chapitre, et c'est une règle d'ordre qui a rang de loi dans cette méthode. **L'équivalence se ferme avant que ne commence toute refactorisation de qualité.** C'est-à-dire, d'abord vous démontrez que le nouveau système est équivalent au vieux, avec sa conception conservatrice initiale, et alors seulement, avec cette équivalence prouvée et au vert, vous commencez à améliorer la qualité interne du nouveau système, à refactoriser, à enrichir, à polir. Pas l'inverse. Jamais l'inverse.

La raison est structurelle, non une préférence. Refactoriser, c'est changer la structure en préservant le comportement. Pour le faire en sécurité, vous avez besoin d'un filet qui vous dise si vous avez cassé le comportement pendant que vous changiez la structure. Ce filet est justement la spécification d'équivalence. Si vous refactorisez avant d'avoir fermé l'équivalence, vous refactorisez sans filet, et un changement qui casse le comportement passe inaperçu, parce que vous n'avez rien pour le détecter. Pire encore, si vous faites les deux choses à la fois, reconcevoir pour l'équivalence et refactoriser pour la qualité mêlées, quand quelque chose échouera vous ne saurez pas s'il a échoué parce que votre reconception n'était pas équivalente ou parce que votre refactorisation a cassé quelque chose. Vous aurez perdu la capacité de diagnostiquer, qui est le même péché que le double saut du chapitre deux, faire deux changements distincts dans le même mouvement.

Cette loi d'ordre a une autorité qu'il vaut mieux incarner dans quelque chose, un gardien qui la fasse respecter. Appelez-le comme vous voulez, ce qui importe c'est sa fonction. C'est l'autorité d'équivalence. Elle possède la spécification, qui est sa preuve. Elle lance les comparaisons. Elle garde les passages au vert, avec leur couverture et leur provenance, pour qu'il soit consigné ce qui a été testé et avec quelle force. Et elle fait respecter l'ordre, elle ne laisse pas commencer la refactorisation de qualité tant que l'équivalence n'est pas fermée. Ce n'est pas seulement un outil qui génère des tests. C'est l'arbitre qui garantit que la méthode se suit dans le bon ordre, parce que l'histoire des désastres de modernisation est pleine d'équipes qui ont refactorisé sur du sable, sur du code qu'elles croyaient équivalent sans l'avoir prouvé, et qui ont découvert en production qu'il ne l'était pas.

Avec toute la vérification debout, chaîne transitive, spécification de domaine comme pivot, parité boîte noire bien calibrée, et la loi d'ordre gardée, vous avez le filet de sécurité qui fait de la reconception de l'ingénierie et non un pari. Mais il manque encore la partie la plus longue et la moins glamour de toute modernisation réelle. Le vieux système ne s'éteint pas le jour où le nouveau est prêt. Les deux cohabitent, parfois durant des années. Bien cohabiter est un art à part entière, et c'est la Partie V.

CardDemo • étape 11/11

Deux champs de CardDemo ressemblent à du mécanisme et n'en sont pas. L'ordre des champs du relevé que les Rapports consomment, et le format du groupe de tarification qui gouverne la recherche du taux, sont lus par quelqu'un en aval, donc ce sont des règles de domaine et ils vont à la comparaison stricte. Le solde du compte après la clôture du cycle se compare jusqu'au dernier centime. Et le cas limite s'écrit comme une ligne de spécification en langage de domaine :

```
Soit un crédit disponible de 100,00 dans le cycle en cours
Quand un mouvement de 100,01 est autorisé
Alors le mouvement est rejeté pour dépassement du crédit du cycle
```

partie V

Cohabiter avec deux systèmes

Chapitre 17

L'étrangleur, honnête

Presque tout ce qui est écrit sur la modernisation traite le passage du vieux au nouveau comme un instant, le jour du changement, comme s'il y avait un interrupteur. Dans la réalité de tout système qui compte, il n'y a pas d'interrupteur. Il y a une période, souvent longue, où le vieux système et le nouveau cohabitent, servant entre eux l'exploitation du métier, pendant que le nouveau assume des responsabilités petit à petit. Concevoir cette cohabitation est un travail d'architecture de premier ordre, non un détail de déploiement, et le sous-estimer est l'une des causes les plus fréquentes qu'une modernisation techniquement correcte finisse en désastre opérationnel. Et la cohabitation n'est pas seulement entre le vieux et le nouveau cœur : le parc qui sort du tri inclut des contextes encapsulés qui ne migrent pas, ils cohabitent pour rester, derrière leur contrat. L'étrangleur gouverne la partie qui se déplace bel et bien, le cœur qui se réécrit.

Le patron classique pour cela est l'étrangleur, que Martin Fowler a nommé d'après un figuier qui pousse autour d'un arbre jusqu'à le remplacer. L'idée est de poser une façade devant tout, qui reçoit toutes les requêtes, et qui pour chacune décide si c'est le vieux système ou le nouveau qui la sert. Au début, presque tout va au vieux. Petit à petit, à mesure que chaque partie du nouveau est prouvée équivalente, la façade commence à router cette partie vers le nouveau. Le figuier grandit. Le vieil arbre s'amenuise. Un jour, le vieux ne sert plus rien et peut être retiré.

Décrit ainsi, cela sonne propre, et la partie routage l'est vraiment. La façade qui décide à qui envoyer chaque requête est, conceptuellement, simple. Le problème, et ici vient l'honnêteté du titre, c'est que router est la partie facile et visible, tandis que la partie difficile et invisible est en dessous, dans les données. Et presque toute la littérature s'enthousiasme pour la partie facile et tait la difficile.

Voyons pourquoi les données sont le problème. Tant que les deux systèmes cohabitent, il y a une donnée métier, mettons l'état d'une entité centrale, que certaines opérations sert déjà le nouveau système et d'autres encore le vieux. Si une opération déjà migrée écrit cette donnée dans le magasin neuf, et qu'une autre non migrée l'écrit dans le magasin vieux, la donnée s'est scindée en deux. Il y a deux versions, dans deux magasins, qui divergent. Le nouveau système croit une chose sur l'entité, le vieux en croit une autre, et le métier est corrompu sans que personne ne le remarque jusqu'à ce qu'il soit trop tard. La façade qui route les opérations ne résout pas cela du tout, parce que le problème n'est pas dans les opérations, il est dans l'état qu'elles partagent.

De là sort le principe qui gouverne le découpage, et il est moins évident qu'il n'y paraît. **Coupez le long de la propriété des écritures, non là où cela vous arrange.** C'est-à-dire, regroupez pour migrer ensemble toutes les opérations qui écrivent une même donnée. Si quatre opérations distinctes écrivent l'état de l'entité centrale, ces quatre migrent en même temps, dans le même pas, ou aucune ne migre. Si vous en migrez deux et en laissez deux dans le vieux, vous scindez l'état et le corrompez. La frontière le long de laquelle vous coupez doit respecter qui est propriétaire d'écrire chaque donnée, parce que c'est ainsi seulement que vous garantissez qu'à chaque instant une donnée est écrite par un seul système, et donc ne peut pas diverger.

Cela explique un ordre de découpage qui autrement paraîtrait arbitraire. Les parties du système qui sont périphériques, qui lisent beaucoup et écrivent peu ou écrivent des données qu'elles seules touchent, sont faciles à migrer en premier, parce que les couper ne scinde aucun état partagé. Les parties centrales, où plusieurs opérations se disputent l'écriture du même état critique, sont celles à migrer en dernier et en bloc, parce qu'elles sont enchevêtrées par la propriété des écritures. Le cœur se migre en entier ou ne se migre pas, et il est laissé pour la fin précisément parce qu'il est le nœud.

Qui tente de migrer le cœur par petits bouts, une opération aujourd'hui et une autre le mois prochain, scinde l'état le plus critique du métier à chaque pas. C'est le chemin le plus court vers le désastre silencieux.

La façade, en outre, fait double service si vous la concevez bien. La même pièce de traduction qui, dans la vérification, convertissait entre le langage du domaine et celui de chaque système sert ici de pont entre les deux systèmes durant la cohabitation. C'est un exemple d'architecture transitoire, au sens du catalogue de déplacement de legacy de Martin Fowler : un échafaudage qu'on installe pour soutenir le remplacement et qu'on retire dès qu'il cesse d'être nécessaire. Ce n'est pas un hasard. C'est l'économie de la méthode récompensant la cohérence. Une pièce bien conçue pour isoler la dépendance à une langue sert aussi bien à vérifier qu'à cohabiter. Nous la reverrons au chapitre suivant, où la cohabitation devient le dernier test avant le changement définitif.

Pour l'instant, la leçon est sobre. L'étrangleur est un excellent patron, mais sa partie célèbre, router, est la facile. La partie qui décide si votre modernisation vit ou meurt est la cohabitation des données, et elle se gouverne par une règle : couper le long de la propriété des écritures, pour qu'aucune donnée n'ait deux propriétaires à la fois. Si vous n'intériorisez qu'une chose de ce chapitre, que ce soit celle-là.

Chapitre 18

La bascule sans foi

Vient le moment de changer pour de vrai une partie du système, de cesser de la router vers le vieux et de la router vers le nouveau de façon définitive. Ce moment s'appelle la bascule (cutover), la coupe. Et la question est, quand est-il sûr de la faire. La réponse paresseuse, quand les tests passent, est insuffisante et dangereuse, parce que les tests passant une fois dans votre environnement ne vous disent pas comment le nouveau système se comportera face au trafic réel, avec toute sa variété et ses cas rares que vos tests ne couvrent peut-être pas.

La méthode propose un critère de bascule qui ne s'appuie pas sur la foi mais sur la preuve accumulée, et il a plusieurs conditions qui doivent tenir en même temps. La première, la spécification d'équivalence est au vert, le nouveau système coïncide avec l'oracle sur tout l'observable de domaine strict. La deuxième, la couverture de cette spécification est suffisante, elle a exercé les branches, les frontières, les cas limites, non seulement le chemin heureux. Une spécification au vert avec une couverture pauvre est une fausse tranquillité, parce qu'elle ne prouve qu'ils coïncident que là où vous avez regardé, et là où vous n'avez pas regardé vivent les bogues. La troisième condition est la décisive et celle que les gens pressés sautent le plus.

La troisième condition est de faire tourner le nouveau système en ombre sur du trafic réel, pendant un temps, et de vérifier qu'il coïncide avec le vieux. Tourner en ombre signifie que le trafic de production réel va aux deux systèmes à la fois, mais seule la réponse du vieux compte, lui qui reste aux commandes. Le nouveau traite le même trafic en parallèle, sa réponse est comparée à celle du vieux, mais elle n'est pas utilisée, elle n'affecte pas le métier. C'est une répétition avec filet, le nouveau agissant sur la réalité sans conséquences. Durant cette période, vous accumulez la preuve que le nouveau coïncide avec le vieux non sur les cas qui vous sont venus à l'esprit, mais sur la distribution réelle de ce qui se passe vraiment, y compris les cas rares que ni votre spécification ni votre imagination n'avaient couverts. L'ombre est là où affleure la perte silencieuse de sémantique qui a échappé à la capture, parce que le trafic réel est l'examineur le plus exhaustif qui soit, il teste tout, sans pitié et sans prévenir.

La quatrième condition ferme le critère. Toute divergence observée en ombre a été enquêtée et résolue, ou ratifiée comme acceptable. Une divergence en ombre, c'est de l'or, c'est le système qui vous avertit d'un cas où le nouveau et le vieux ne coïncident pas. Chacune mérite enquête. Presque toujours elle révèle une faute du nouveau que vous corrigez. Parfois elle révèle que la divergence est voulue, un changement de règle que vous avez décidé exprès, et alors elle se ratifie et se documente comme telle. Ce qui ne se fait jamais, c'est de l'ignorer. Une bascule avec des divergences d'ombre inexpliquées est une bascule avec foi, justement ce que la méthode rejette.

Il y a une leçon historique derrière toute cette prudence, et le chapitre un lui a mis des noms. Les modernisations qui ont fini en gros titres de désastre, des systèmes qui ont massivement corrompu des données après le changement, partagent presque toujours la même racine. Ce ne fut pas une faute d'outil. Ce fut une faute de gouvernance. Quelqu'un a affirmé l'équivalence au lieu de la démontrer, a basculé sur cette affirmation, et la réalité a encaissé la différence. Le critère de bascule sans foi, gate au vert plus couverture plus ombre plus divergences résolues, est la réponse directe à cette classe de désastre. Vous ne basculez pas sur ce que vous croyez. Vous basculez sur ce que vous avez démontré et confirmé contre la réalité.

Il y a une cinquième condition, et le cimetière du chapitre un la justifie à lui seul. La coupe a besoin d'une marche arrière répétée : un chemin de retour vers le vieux système qui a été exercé, pas seulement écrit, avec ses données et ses délais

clairs. Le désastre de la paie australienne n'a pas été seulement de mal couper ; ç'a été de couper sans issue. **La coupe est réversible jusqu'à ce qu'elle cesse de l'être, et cette frontière se franchit sciemment, elle ne se découvre pas.** Tant que le retour existe, une surprise est un incident ; quand il cesse d'exister, la même surprise est un désastre. Savoir à quel point exact du plan ce navire se brûle, et décider de le franchir sur des preuves et non sur un calendrier, fait partie du critère.

Et une honnêteté de plus, qui rejoint la première leçon du cimetière. Ce livre se concentre sur l'équivalence de comportement, mais la coupe déplace aussi de l'état : la charge initiale des données, leur réconciliation contre la source, les comptages qui tombent juste. Cette discipline a son propre métier et cette méthode ne la détaille pas, mais elle l'exige au gate : la réconciliation des données au vert est une condition de la coupe autant que la parité au vert, parce que la parité des données et celle du comportement sont deux preuves distinctes, et le cimetière est plein de qui a présenté l'une comme si c'était l'autre.

Quand toutes les conditions tiennent, la coupe peut se faire, et il vaut mieux la faire graduelle même alors. Au lieu d'envoyer d'un coup tout le trafic au nouveau, vous lui en envoyez une petite fraction comme système aux commandes, vous surveillez, et s'il tient, vous montez la fraction petit à petit. C'est la prudence portée jusqu'au dernier mètre. Même avec toute la preuve accumulée, un changement graduel vous laisse une issue si quelque chose d'inattendu apparaît avec le nouveau système vraiment aux commandes.

Un avertissement technique honnête pour ne pas vendre l'ombre comme gratuite. Tourner en ombre exige que le nouveau système traite du trafic réel sans causer d'effets réels dans le monde. Ses écritures doivent aller à un magasin d'ombre, non à celui de production. Ses effets externes, envoyer une notification, appeler un tiers, doivent être supprimés, parce que sinon la répétition devient réalité. Pour des opérations de pur calcul et de changement d'état interne, c'est propre. Pour des opérations à effets externes visibles, c'est plus délicat, vous ne pouvez pas vraiment envoyer la notification deux fois, donc vous supprimez l'effet externe et ne comparez que la décision interne qui l'aurait déclenché. Bien concevoir cette suppression fait partie du coût de l'ombre, et il vaut mieux le budgéter que le découvrir tard.

Avec la bascule faite, partie par partie, le nouveau système prend les commandes et le vieux se retrouve sans travail. Vient la dernière question de la cohabitation, et elle est plus subtile qu'il n'y paraît. Comment mettre le vieux système à la retraite sans perdre ce que lui seul savait.

Chapitre 19

Mettre l'oracle à la retraite sans perdre la référence

Le vieux système, et sa réplique fidèle qui a fait office d'oracle, ont été votre source de vérité durant toute la modernisation. Chaque fois que vous doutiez de ce que devait faire le nouveau système, vous le demandiez à l'oracle. Maintenant le nouveau est aux commandes et le vieux est superflu. La tentation est de l'éteindre et de fêter ça. Mais éteindre l'oracle, c'est perdre pour toujours la seule source qui savait, avec certitude, ce que faisait le vieux système dans chaque cas, y compris ceux que vous n'avez jamais capturés. Il faut le mettre à la retraite avec soin, parce que c'est un soin qui ne se répète pas, une fois éteint, il n'y a pas de retour.

La règle de séquence est tranchante. **Capturez tout ce dont vous aurez besoin de l'oracle avant de le mettre à la retraite, parce qu'ensuite vous ne pourrez pas le re-dériver.** Tant que l'oracle vit, vous pouvez lui poser n'importe quelle question nouvelle qui vous vient et obtenir la vérité. Quand il meurt, la seule référence qui reste est ce que vous avez capturé, la spécification gelée. Si un cas n'est pas dans la spécification et que l'oracle n'existe plus, ce comportement s'est perdu comme source de vérité, il ne vous reste que ce que fait le nouveau système, que vous ne pouvez plus confronter à rien. C'est pourquoi la mise à la retraite de l'oracle est gardée par une condition, la spécification capture tout ce dont nous aurons besoin, à la couverture dont nous avons convenu qu'elle est suffisante. Tant que cette condition ne tient pas, l'oracle ne s'éteint pas.

Une fois tout capturé et l'oracle mis à la retraite, la spécification gelée devient la référence permanente, et ici on voit pourquoi nous avons tant insisté sur sa forme. La spécification, qui était un patron de parité tant que l'oracle vivait, devient le filet de régression du nouveau système, désormais seulement avec sa pièce de traduction, sans celle du vieux qui n'existe plus. Chaque changement futur du nouveau système se vérifie contre cette spécification gelée, qui garantit que le comportement hérité est préservé. Et si jamais il faut changer délibérément une règle héritée, ce changement se fait en éditant des lignes concrètes de la spécification, de façon explicite et ratifiée, non par accident. La spécification est la mémoire du vieux système, écrite de façon à survivre à son auteur.

Il y a une nuance pratique de coût qu'il vaut mieux exploiter, et elle distingue deux mises à la retraite que les gens confondent. Une chose est de mettre à la retraite le vieux système pour de vrai, l'original, celui qui tournait sur la plateforme chère, et cela est cher et se fait à la bascule définitive, libérant le coût qui justifiait toute la modernisation. Autre chose, très différente, est de mettre à la retraite la réplique fidèle, l'oracle local, qui tourne à bas coût sur votre propre infrastructure. La réplique est bon marché à garder allumée. Donc une stratégie prudente est de retirer le vieux système original dès que la bascule le permet, pour capturer l'économie, mais de conserver la réplique fidèle un temps de plus, comme oracle de réserve. Si après la bascule une surprise apparaît en production, un cas que le nouveau système gère de façon douteuse, vous pouvez redemander à la réplique ce qu'aurait fait le vieux, et désambiguïser. La réplique est l'assurance bon marché que vous conservez après avoir éteint la chère, et vous la mettez à la retraite vous aussi quand la confiance accumulée fait qu'elle n'en vaut plus la peine.

Il faut clore par le résidu honnête, parce que cette méthode ne promet pas la certitude absolue et feindre qu'elle la promet trahirait tout ce qui a été dit. Quand l'oracle, original et réplique, n'existe plus, la spécification est la seule référence, et la spécification est finie. Tout comportement du vieux système qui n'a pas été capturé, les chemins qui ne se sont jamais

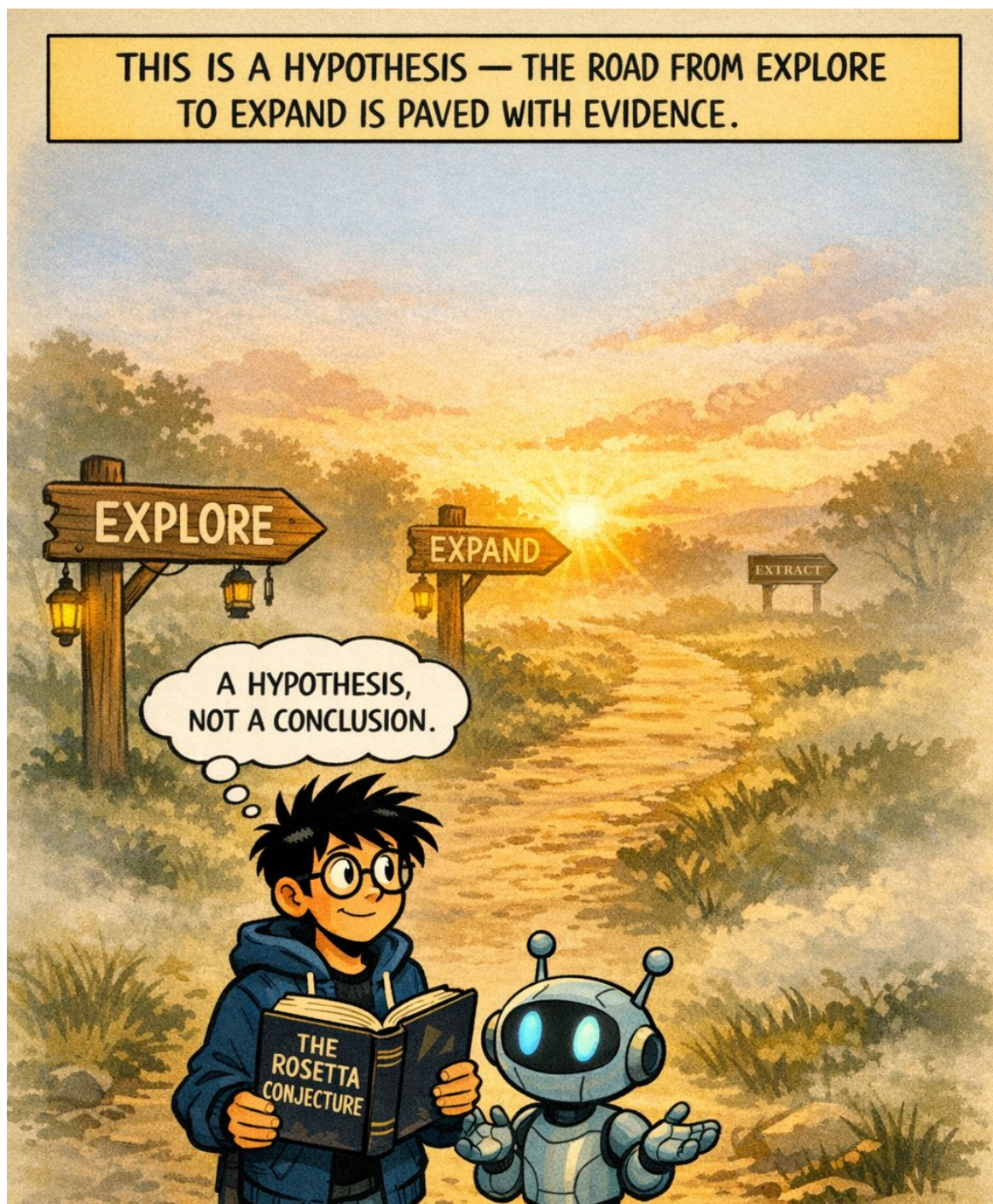
exercés, les cas rares que même le trafic d'ombre n'a pas déclenchés, est devenu irrécupérable depuis sa source originale. Si l'un de ces cas se réveille un jour en production, il n'y a plus d'oracle à qui demander ce qui était correct, il ne reste qu'à le décider à nouveau, désormais comme conception, non comme récupération. Il entre par la porte des bogues et se traite comme un de plus : on le reproduit, on décide ce qu'il doit faire, on le spécifie et on le couvre, comme tout défaut qui apparaît en production. Ce n'est pas un échec de la méthode, c'est une limite de la réalité, aucun processus ne peut capturer ce qui ne s'est jamais manifesté. Ce que la méthode fait, c'est borner ce résidu et le rendre visible, capturer large tant que l'oracle vit, faire tourner l'ombre sur du trafic réel pour traîner les cas rares à la lumière, et marquer honnêtement les trous qui restent. Après avoir mis l'oracle à la retraite, la réalité est le seul maître qui reste. La méthode s'assure que vous arriviez à ce point ayant appris de l'oracle tout ce qui pouvait s'apprendre.

partie VI

D'Explore à Expand

Chapitre 20

Ceci est une hypothèse



Arrivés ici, il convient d'ôter un masque que bien des livres techniques n'ôtent jamais. Ce que vous venez de lire n'est pas une méthode éprouvée contre des centaines de migrations de production. C'est une hypothèse, cohérente et raisonnée, sur la manière dont la modernisation devrait se faire quand on prend au sérieux qu'elle est un problème de domaine et qu'on tire parti du fait qu'il y a désormais des agents à tous les étages de l'ascenseur. Elle est, dans les termes de Beck, dans la phase Explore. Et le dire n'affaiblit pas le livre, cela le renforce, parce qu'un argument présenté comme ce qu'il est résiste mieux qu'une promesse présentée comme ce qu'elle n'est pas.

Il vaut la peine de rassembler pourquoi la méthode devrait fonctionner, car c'est la substance de l'hypothèse. Elle devrait fonctionner parce qu'elle attaque la cause réelle de l'échec, confondre modernisation et traduction, au lieu d'un symptôme. Elle devrait fonctionner parce qu'elle s'appuie sur une source de vérité solide, le comportement observable du système vivant, au lieu d'une documentation qui ment ou d'un code qui trompe. Elle devrait fonctionner parce qu'elle préserve ce qui importe vraiment, la sémantique, et libère ce qui gêne vraiment, la vieille structure, au lieu de confondre les deux. Elle devrait fonctionner parce qu'elle démontre l'équivalence au lieu de la supposer, et elle la démontre par une chaîne de pas chacun facile à vérifier. Et elle devrait fonctionner parce qu'elle traite la cohabitation et le changement avec le respect qu'ils méritent, au lieu de feindre qu'un interrupteur existe. Rien de tout cela, je ne l'invente. Chaque pièce repose sur des épaules connues qui ont survécu à leur propre épreuve du temps : la conception dirigée par le domaine d'Eric Evans, l'ascenseur de l'architecte de Gregor Hohpe, le 3X de Kent Beck, l'étrangleur de Martin Fowler et le travail de Michael Feathers sur la manière de comprendre et de changer les systèmes hérités. Bien moderniser un système qui compte fut toujours difficile. Ce qui a changé, c'est qu'avec un agent à chaque étage de l'ascenseur, c'est enfin abordable. Le pari du livre est que les combiner ainsi, avec le domaine au centre et l'agent comme augmentation à chaque étage, produit quelque chose de plus que la somme, un processus qui gagne le mot modernisation.

Mais une hypothèse n'est pas une conclusion, et il vaut mieux être clair sur ce qui la transformerait en quelque chose de plus. Passer d'Explore à Expand exige une preuve qui, à l'heure actuelle, n'existe pas. Cela exige de parcourir la méthode entière sur des mandats réels de production, plusieurs, différents les uns des autres, et de voir non seulement qu'elle fonctionne mais de comprendre pourquoi elle fonctionne assez bien pour prédire quand elle ne fonctionnera pas. Cette capacité à prédire l'échec est le signe de maturité de Beck, la frontière entre une intuition heureuse et une méthode. Tant que vous ne l'avez pas, l'attitude honnête est de continuer à explorer, de parcourir l'ascenseur par cycles rapides, d'apprendre de chacun, de jeter sans regret ce qui se révèle faux, et de résister à la tentation de figer et de productiser trop tôt.

Il y a des questions ouvertes que la méthode elle-même ne résout pas et que la phase Expand devra aborder. Comment la récupération de domaine monte-t-elle à l'échelle quand le système n'est pas un seul mais une constellation de systèmes enchevêtrés entre eux ? Que se passe-t-il quand le domaine récupéré révèle que le métier lui-même ne veut plus faire ce que le vieux système faisait, quand la modernisation découvre une occasion de changer les règles et non seulement de les préserver ? Où est la limite réelle de ce qu'un agent peut récupérer avec fiabilité avant que le jugement humain n'ait à porter presque tout ? Comment gouverne-t-on tout cela quand il n'y a pas un architecte mais plusieurs, aux critères différents, sur le même système ? Aucune de ces questions n'a ici de réponse close, et feindre qu'elle l'a serait remettre le masque. Ce sont le travail qui reste.

Cette honnêteté est, au final, cohérente avec la méthode elle-même. Tout le livre prêche qu'il vaut mieux démontrer que supposer, que la provenance importe, qu'il faut marquer ce qu'on ne sait pas au lieu de feindre la certitude. Il serait incohérent que le livre s'exempte de sa propre discipline et se présente comme vérité prouvée alors qu'il est hypothèse raisonnée. Voici donc sa provenance, marquée comme la méthode l'exige. Ceci est un argument à partir de principes, non une trouvaille de terrain validée. Sa force est dans la solidité du raisonnement et dans la qualité des principes qu'il combine, non dans une preuve qui n'existe pas encore.

Si la méthode est bonne, le parcours le démontrera, mandat après mandat, cycle après cycle, dans la phase Explore où elle vit maintenant, jusqu'à ce que la preuve justifie de la mettre à l'échelle. Et si dans ce parcours elle se révèle fautive en quelque partie, la bonne attitude ne sera pas de la défendre, ce sera d'apprendre de l'échec et de la refaire, parce que c'est exactement ce que veut dire explorer. Ce livre n'est pas la fin d'une investigation. C'est la meilleure forme que j'ai su donner, aujourd'hui, à une question encore ouverte. Comment moderniser pour de vrai un système qui compte, en mettant le domaine au centre, avec un agent à chaque étage de l'ascenseur, et sans rien casser en chemin. Je ne connais aucune autre tentative de se planter à cette intersection, le domaine, les agents et la modernisation, en ne promettant que ce qui peut se démontrer. Si une meilleure apparaît, qu'elle soit la bienvenue, car la question importe plus que celui qui y répond. L'invitation est à le parcourir, non à le croire.

Le domaine dirige. L'agent rend la prudence abordable.

Annexe A

Antipatrons

Les antipatrons sont les erreurs que la méthode combat, rassemblées ici pour les reconnaître vite. Chacun est l'ombre d'un principe du livre.

La traduction qui se croit modernisation. Transpiler le vieux système dans le nouveau langage ligne par ligne et déclarer le travail fait. Elle conserve la vieille conception et le mécanisme entrelacé, désormais en syntaxe moderne et donc plus difficile à repérer. C'est l'antipatron racine, celui qui donne sens à tout le livre.

La vitesse sur un pipeline non certifié. Accélérer avec des agents une chaîne de traduction qui n'a jamais prouvé l'équivalence de comportement. La vitesse appliquée à une méthode qui ne fonctionne pas ne produit pas de modernisation, elle produit le prochain post-mortem plus tôt.

La recompilation qui se croit destination. Faire tourner le vieux système sur une plateforme neuve sans toucher à sa conception, et croire qu'avec ça on a modernisé. C'est une relocalisation légitime déguisée en transformation. L'erreur n'est pas de recompiler, c'est de croire qu'on a fini.

Tuer le COBOL trop tôt. Réécrire ou transpiler du code qui était stable, correct et faisait son travail, au lieu de l'encapsuler et de le laisser tranquille. Cela dépense du budget et des experts rares sur ce qui ne le demandait pas, et échange l'optionnalité contre du risque.

La refactorisation d'abord, le filet après. Commencer à améliorer la structure du nouveau système avant d'avoir démontré son équivalence avec le vieux. Refactoriser sans filet, sans rien pour détecter si on a cassé le comportement. Cela viole la loi d'ordre, et c'est la racine des désastres de modernisation les plus retentissants.

Le double saut. Changer la plateforme et la conception dans le même mouvement. Quand quelque chose échoue, et quelque chose échoue, vous ne pouvez pas diagnostiquer si c'était le changement de plateforme ou celui de conception. Vous perdez la capacité d'isoler la cause juste quand vous en avez le plus besoin.

Le choix déguisé en trouvaille. Glisser une décision de conception dans le modèle récupéré comme si c'était quelque chose d'observé dans le vieux système. Cela contamine le portrait du domaine, parce que cela mêle ce que le système fait avec ce que vous voulez qu'il fasse. La discipline de provenance existe pour le détecter.

La ratification qui résout les doutes. Transformer, dans une réunion de ratification, une conjecture douteuse en fait accepté, par consensus et sans preuve nouvelle. La ratification doit filtrer ce qui a une source ferme, non bénir le douteux d'une signature.

La perte silencieuse de sémantique. Jeter comme mécanisme technique quelque chose qui cachait une règle métier. Le cas typique est la gestion d'une erreur technique qui en réalité encodait une décision de domaine. Là où le mécanisme et la logique s'entrelacent, là rôde cet antipatron, et seul l'oracle le désambiguïse.

La parité qui compare de trop. Inclure dans la comparaison d'équivalence des détails techniques que le nouveau système a le droit de faire autrement. Cela produit une mer de faux positifs, le test devient du bruit, et l'équipe finit par l'ignorer, ce qui est le pire qui puisse arriver à un test.

La parité qui compare de trop peu. Exclure de la comparaison des choses qui étaient bel et bien des observables de domaine. Cela produit une fausse sécurité, laisse passer de vrais changements de comportement, et vous invite à casser le système la conscience tranquille. Pire que le précédent, parce que le silence trompe plus que le bruit.

Le découpage qui scinde l'état. Migrer par petits bouts un ensemble d'opérations qui écrivent la même donnée, en laissant les unes dans le vieux système et les autres dans le nouveau. La donnée se scinde en deux magasins qui divergent. Cela corrompt l'état le plus critique du métier sans prévenir. On l'évite en coupant le long de la propriété des écritures.

La bascule avec foi. Basculer vers le nouveau système parce que les tests passent, sans avoir tourné en ombre sur du trafic réel ni résolu les divergences. C'est affirmer l'équivalence au lieu de la démontrer contre la réalité. L'histoire des désastres de modernisation est écrite en grande partie avec cet antipatron.

Éteindre l'oracle trop tôt. Mettre à la retraite le vieux système et sa réplique avant d'avoir capturé tout ce dont on allait avoir besoin. Une fois éteint, il n'y a pas de retour, et le comportement non capturé se perd comme source de vérité pour toujours.

Productiser en Explore. Prendre une méthode ou une approche qui a fonctionné une fois et la figer en standard d'entreprise avant de la comprendre assez pour prédire quand elle échouera. Cela saute d'Explore à Extract sans passer par Expand. Un succès n'est pas un patron.

Annexe B

Glossaire

Agent. Système capable de lire une grande quantité de matière, de raisonner dessus et de produire des résultats. Dans ce livre, une augmentation de l'architecte présente à tous les étages de l'ascenseur, qui trouve mais ne choisit pas.

Harnais. La machine à états avec gardes qui organise le travail de l'agent. Chaque état a son but, son contexte propre et son accès aux outils borné, et une transition n'a lieu que si une condition déterministe tient.

Ascenseur de l'architecte. Métaphore de Gregor Hohpe. L'architecte monte et descend entre le penthouse, où se décide la stratégie, et la salle des machines, où le code s'exécute, connectant des étages qui sinon ne se parleraient pas.

Chaîne transitive. Stratégie de vérification qui évite de comparer le nouveau système au vieux en direct. Elle démontre des équivalences intermédiaires faciles, vieux avec réplique fidèle, réplique avec spécification, spécification avec nouveau système, et laisse la transitivité conclure l'équivalence finale.

Couche anticorruption (ACL). Patron stratégique du DDD. Couche qu'un contexte consommateur dresse pour traduire dans son propre modèle celui de tout autre, afin que la forme étrangère ne le contamine pas. Dans ce livre, son usage le plus fréquent est de protéger le nouveau modèle de la forme du vieux système.

Contexte délimité (bounded context). Frontière à l'intérieur de laquelle un modèle du domaine et son langage sont cohérents. Les coutures entre contextes sont là où l'on peut plus tard couper.

Bascule (cutover). La coupe. Le moment où une partie du système cesse d'être servie par le vieux et passe à être servie par le nouveau de façon définitive.

Conception dirigée par le domaine (DDD). Approche qui modélise le logiciel à partir du domaine métier et de son langage, avec des concepts comme invariants, agrégats, contextes délimités et langage omniprésent.

Double saut. Antipatron consistant à changer plateforme et conception dans le même mouvement. Interdit parce qu'il détruit la capacité de diagnostiquer les échecs.

Spécification exécutable. Collection de cas sous la forme étant donné ceci, quand cela, alors ceci, écrits en langage de domaine, qu'une machine peut exécuter et vérifier. C'est le pivot de la chaîne transitive et elle vit trois vies, parité, régression et documentation.

Fidélité. Engagement de préserver les invariants et la sémantique du vieux système, non sa structure. Ce que le système signifie est sacré, comment il y parvient est libre.

GraphRAG. Appariement d'une recherche sémantique qui propose des candidats et d'un graphe déterministe qui les vérifie, sur une projection du système. La recherche propose, le graphe dispose.

Invariant. Règle que le domaine doit toujours satisfaire. Part de l'espace du problème, elle survit à toute reconception.

Langage publié (PL). Patron stratégique du DDD. Langage partagé et bien documenté, généralement associé à un hôte ouvert, qui sert de medium commun dans lequel deux contextes communiquent. C'est la langue du contrat, non le contrat lui-même.

Langage omniprésent. Vocabulaire partagé d'un contexte du domaine, dans lequel s'exprime la spécification, indépendant de la langue de tout système concret. Il y en a un par contexte délimité, non un global. Son indépendance est ce qui protège la liberté de reconception.

Carte des contextes. Patron stratégique du DDD. Le plan des contextes délimités d'un parc et des relations entre eux. Dans cette méthode il y en a deux : la carte as-is récupérée, qui est une donnée, et la carte cible, qui est une décision.

Noyau partagé (Shared Kernel, SK). Patron stratégique du DDD. Sous-ensemble du modèle que deux contextes partagent et maintiennent en commun, avec une coordination étroite. Il couple, on le réserve donc à ce qui doit vraiment être identique des deux côtés.

Client et fournisseur (Customer/Supplier). Patron stratégique du DDD. Relation entre deux contextes où celui d'aval est client et celui d'amont fournisseur, négociant priorités et engagements.

Conformiste (Conformist). Patron stratégique du DDD. Le contexte d'aval adopte le modèle d'amont tel quel, sans traduire. Bon marché et honnête quand le modèle d'autrui est bon ; le contraire de lever une couche anticorruption.

Voies séparées (Separate Ways). Patron stratégique du DDD. Décision de ne pas intégrer deux contextes et de dupliquer le peu qu'il faut, quand intégrer coûte plus que ça ne vaut.

Mécanisme. La plomberie qui fait fonctionner la logique sur une plateforme concrète. Espace de la vieille solution. C'est ce que vous avez le droit de changer, face à la logique métier, que vous devez préserver.

Modèle récupéré. Portrait fidèle du domaine du vieux système, en termes de DDD, dépouillé du mécanisme, avec chaque affirmation marquée par sa provenance. Il décrit l'état actuel du domaine, non l'état désiré.

Oracle. Le vieux système, ou sa réplique fidèle, en tant que source de vérité sur le comportement. Vous lui donnez une entrée, il vous donne la sortie correcte par définition. Il remplace la documentation et le souvenir comme source.

Parité boîte noire. Vérification d'équivalence qui ne compare que l'observable depuis l'extérieur, entrées et sorties de domaine, jamais les entrailles, parce que les entrailles divergent exprès.

Parité de comportement. Ce que la vérification poursuit : que le nouveau système signifie la même chose que l'ancien dans l'observable du domaine. Ce n'est pas la même chose que la parité des données, déplacer les données intactes, qui peut s'obtenir pendant que le comportement se casse.

Pipeline non certifié. Chaîne de transformation, aujourd'hui souvent agentique, qui livre du code neuf sans avoir démontré l'équivalence de comportement contre la réalité. C'est la critique du livre à l'industrie, qui s'applique à Rosetta elle-même tant que l'équivalence n'est pas démontrée de bout en bout.

Provenance. Marque qui déclare d'où vient chaque affirmation du modèle récupéré, et donc combien on peut s'y fier. Extrait, inféré, attesté, non prouvé. Ce qui ne peut citer de source est un choix, non une trouvaille, et quitte le modèle récupéré.

Réplique fidèle. Copie du vieux système qui préserve sa structure et tourne vite en local, faite par exemple en recompilant vers une plateforme moderne. Elle sert d'oracle bon marché et rapide. À ne pas confondre avec le nouveau système, qui diverge exprès.

Signaux. Ce que le système émet sur lui-même pendant qu'il travaille, avec sa provenance, et qui permet aux gardes de décider sur des bases solides. Cela inclut la couverture, qui met un chiffre sur le résidu.

Service de host ouvert (Open Host Service, OHS). Patron stratégique du DDD. Protocole ou service public par lequel un contexte offre ses capacités à de nombreux consommateurs à la fois, sans exposer son intérieur. C'est la façon d'encapsuler le COBOL stable : on l'invoque, on n'en hérite pas.

Sur la boucle (on-the-loop). Position de l'architecte face au harnais. Il n'approuve pas chaque transition, ce sont les gardes qui le font, mais il surveille la machine et décide là où, par conception, la machine ne décide pas. Par opposition à être dans la boucle.

Ombre. Mode de cohabitation où le trafic réel va aux deux systèmes à la fois, mais seule la réponse du vieux compte. Le nouveau traite en parallèle sans affecter le métier, et sa réponse est comparée. C'est le dernier test contre la réalité avant la bascule.

Substrat déterministe. La projection du système dérivée mécaniquement, reproductible et avec provenance, sur laquelle l'agent raisonne et contre laquelle on vérifie. Elle n'est pas inventée par l'agent.

Triage. Décision de portefeuille, postérieure à la récupération, sur ce qu'il faut faire de chaque partie du système selon son type de sous-domaine : le générique à un produit, le soutien et le stable à encapsuler, le cœur à réécrire.

Étrangleur. Patron de Martin Fowler. Une façade devant tout route chaque requête vers le vieux ou le nouveau, et déplace peu à peu les responsabilités vers le nouveau, jusqu'à ce que le vieux puisse être retiré.

3X. Modèle de Kent Beck. Explore, Expand, Extract. Trois phases dans la vie d'une idée, chacune avec son état d'esprit. Les confondre est une erreur chère. Cette méthode vit dans Explore.